

ICPC Team Notebook

UIT.BTC

University of Information Technology, VNU-HCM

Vu Gia Bao, Nguyen Khanh Tai, Huynh Trung Cuong

Contents

1	Data Structure	1
1.1	Iterative Segment Tree	1
1.2	Lazy Segment Tree (Implementation Tips)	2
1.3	Sparse Segment Tree	2
1.4	Cartesian Tree	2
1.5	Persistent Segment Tree	2
1.6	Parallel Binary Search	3
1.7	Wavelet Tree	3
1.8	Segment Tree Beat	3
1.9	Kd Tree	3
1.10	BIT2D	4
1.11	Treap	4
1.12	Splay Tree / Link - cut Tree	4
1.13	Farach - Colton - Bender	5
1.14	Trie	5
1.15	Lichao Tree	6
1.16	Line Container	6
1.17	Arpa Trick	6
1.18	Blocky Data Structure	6
1.19	Ultimate Hashmap	7
1.20	Erasable Priority Queue	7
1.21	Expandable Data Structure	7
1.22	Interval Set	7
2	Graph	7
2.1	Minimum Cost Flow	7
2.2	MinCut - MaxFlow	7
2.3	Bipartite Matching	8
2.4	Hungarian Matching	8
2.5	General Matching	8
2.6	Biconnected Component	9
2.7	Functional Cycle	9
2.8	Strongly Connected Components	9
2.9	Offline Deletion	9
2.10	2-SAT	9
2.11	Floyd	9
2.12	Eulerian Path	10
2.13	Vizing Theorem	10
2.14	Bipartite Online Checking	10
2.15	Gomory-Hu Tree	10
2.16	Critical (Dominator Tree)	10
2.17	Prufer	10
2.18	Steiner Tree	11
2.19	Stable Marriage	11
3	Tree	11
3.1	Tree Isomorphism	11
3.2	Centroid Decomposition	11
3.3	Heavy-Light Decomposition	11
4	String	11
4.1	String Problem Solving Strategies	11
4.2	Rolling Hash (61-bit Mersenne)	12
4.3	KMP	12
4.4	Z-Algorithm	12
4.5	Aho Corasick	12
4.6	Suffix Array	12
4.7	Suffix Automaton	13
4.8	Lyndon Factorization	13
4.9	Manacher	13
4.10	Palindrome Tree	13
4.11	De Bruijn	13

5	Dynamic Programming	13
5.1	Reroot DP	13
5.2	Divide-And-Conquer DP & Quadrangle Inequality	14
5.3	1D/1D DP Optimization	14
5.4	Alien DP	14
5.5	Convex Hull Trick	14
5.6	Slope Trick	15
5.7	SOS DP	15
5.8	Monotonic Queue Optimization	15
5.9	Expected Value Trick	15
6	Math	15
6.1	Number Theory Formulas	15
6.2	Like-Euclid Floor Sum	15
6.3	Rabin-Miller	15
6.4	Fast Sieve	16
6.5	Prime Counting	16
6.6	Pollard Rho	16
6.7	Primitive Root	16
6.8	Baby-step Giant-Step	16
6.9	Sqrt Mod	16
6.10	Extended GCD	16
6.11	Diophantine	17
6.12	Stern-Brocot Tree	17
6.13	Multiplicative Functions & Möbius Inversion	17
6.14	Combinatorics Formulas	17
6.15	Generating Functions & Taylor Series	17
6.16	Linear Recurrences (Berlekamp-Massey)	17
6.17	FWHT	18
6.18	Lagrange	18
6.19	FFT, NTT, and Polynomials	18
6.20	Linear Algebra (Gauss)	19
6.21	XOR Linear Basis	19
6.22	Burnside Lemma & Pólya Theorem	19
6.23	Game Strategy & State Reasoning	19
6.24	Probability & Expected Value	20
6.25	Bignum	20
7	Geometry	20
7.1	Primitive Geometry	20
7.2	Convex Hull	20
7.3	Rotating Calipers	20
7.4	Minkowski Sum	21
7.5	Point in Convex Polygon	21
7.6	Line Hull Intersection	21
7.7	Polygon Raycast	21
7.8	Half-plane Intersection	21
7.9	Smallest Enclosing Circle	21
7.10	Circle Tangents	21
7.11	Geometry Formulas	22
7.12	AREA	22
7.13	Manhattan MST	22
7.14	Planar Dual Graph	22
8	Misc	22
8.1	Template	22
8.2	Debugging Magic	23
8.3	Expression Parsing	23
8.4	Johnson Scheduling	23
8.5	Matrix Multiplication Check	23
8.6	Calendar	23
8.7	Josephus	23
8.8	Interactive Tricks	23
8.9	Random, Bit Tricks, Bitset	24
8.10	Transformations	24
8.11	Checked	24

1 Data Structure

1.1 Iterative Segment Tree

Complexity: $\mathcal{O}(\log n)$ per query/update, $\mathcal{O}(n)$ build
Fast non-recursive segment tree with point update, range query, and Dual SegTree (Range update + Point query)

```
1 // Iterative SegTree (0-indexed [l, r], Point update, Range max
  query, Tree Walk, Dual)
2 const int N = 2e5 + 5;
3 int n, sz, tree[4 * N];
4
5 void init(int _n) {
6     n = _n; sz = 1;
7     while (sz < n) sz <= 1;
8     fill(tree, tree + 2 * sz, 0);
9 }
10 void build(int a[]) {
11     init(n);
12     for (int i = 0; i < n; i++) tree[sz + i] = a[i];
13     for (int i = sz - 1; i > 0; i--) tree[i] = max(tree[i << 1],
        tree[i << 1 | 1]);
14 }
15 void update(int p, int val) { // a[p] = val
16     for (tree[p += sz] = val; p > 1; p >= 1) tree[p >> 1] = max(
        tree[p], tree[p ^ 1]);
17 }
18 int query(int l, int r) { // max [l, r]
19     int res = 0;
20     for (l += sz, r += sz + 1; l < r; l >= 1, r >= 1) {
21         if (l & 1) res = max(res, tree[l++]);
22         if (r & 1) res = max(res, tree[--r]);
23     }
24     return res;
25 }
26 // Tree Walk O(log N): First index i >= l with tree[i] >= val (or n
  if none)
27 int max_right(int l, int val) {
28     int cur = 0;
29     for (l += sz; ; l >= 1) {
30         if (l & 1) {
31             if (max(cur, tree[l]) >= val) {
32                 while (l < sz) {
33                     l <= 1;
34                     if (max(cur, tree[l]) < val) cur = max(cur,
                        tree[l++]);
35                 }
36                 return min(n, l - sz);
37             }
38             cur = max(cur, tree[l++]);
39         }
40         if ((l & (l - 1)) == 0) break;
41     }
42     return n;
43 }
44 // Tree Walk O(log N): Last index i <= r with tree[i] >= val (or -1
  if none)
45 int min_left(int r, int val) {
46     int cur = 0;
47     for (r += sz + 1; ; r >= 1) {
48         if (r & 1) {
49             --r;
50             if (max(tree[r], cur) >= val) {
51                 while (r < sz) {
52                     r = r << 1 | 1;
53                     if (max(tree[r], cur) < val) cur = max(tree[r
                        --], cur);
54                 }
55             }
56         }
57     }
58 }
```

```

55         return r - sz;
56     }
57     cur = max(tree[r], cur);
58 }
59 if ((r & (r + 1)) == 0) break;
60 }
61 return -1;
62 }
63 // Dual SegTree: Range update [l, r] += val, Point query at p
64 void update_range(int l, int r, int val) {
65     for (l += sz, r += sz + 1; l < r; l >= 1, r >= 1) {
66         if (l & 1) tree[l++] += val;
67         if (r & 1) tree[--r] += val;
68     }
69 }
70 int query_point(int p) {
71     int res = 0;
72     for (p += sz; p > 0; p >= 1) res += tree[p];
73     return res;
74 }

```

1.2 Lazy Segment Tree (Implementation Tips)

Lazy propagation discipline, tag compositions (Affine, Range Set), and $O(\log n)$ Tree Walk

- **Push/Pull Discipline:** Call `push(id, l, r)` at the start of `update` / `query`; call `pull(id)` (`st[id] = merge(st[2*id], st[2*id+1])`) after updating children.

- **Multiple Lazy Tags:**

- **Affine** ($x \mapsto a \cdot x + b$): Composing new (c, d) on $(a, b) \implies c(ax + b) + d = (ca)x + (cb + d)$.
Tag: `laz_a = laz_a * c; laz_b = laz_b * c + d;`
Node: `val = val * c + (r - l + 1) * d;`
- **Range Set vs Range Add:** Range Set ($x \leftarrow v$) overrides previous Range Add $\implies$ `reset add = 0.`

- **Tree Walk** ($O(\log n)$): If left child satisfies condition ($\max \geq X$ or $\text{sum} \geq S$), push lazy and recurse left; else adjust target and recurse right.

- **Bounds:** Allocate $4N$ for 1-based indexing (`id < 1` and `id < 1 | 1`).

1.3 Sparse Segment Tree

Complexity: $O(\log n)$ per query

Dynamic Segment Tree allocating nodes on demand

```

1 struct Node{
2     int l, r, ll, rr; // [l, r] are the ranges | ll, rr: left, right
        children
3 };
4 struct sparseIT{
5     /*A sample sparse IT with following operations:
6     - Get the number of elements in a range.
7     - Get the sum of elements in a range.
8     Lazy operations could be implemented as well
9     */
10    vector<Node> st;
11    int rt = 0;
12    int getInfo(int nn){
13        //Return Some kind of info
14    }
15    void update(int nn){
16        //Update the node nn.
17    }
18    sparseIT(int _l = 0, int _r = 0){

```

```

19    rt = 0;
20    st.pb(Node(_l, _r));
21 }
22 void updatePos(int nn, int pos, int value){
23     if (st[nn].l == st[nn].r){
24
25         //Change update position
26         return;
27     }
28     int mid = (st[nn].l + st[nn].r) >> 1;
29     if (pos <= mid){
30         if (st[nn].ll == -1) { //init
31             st[nn].ll = st.size();
32             st.pb(Node(st[nn].l, mid));
33         }
34         updatePos(st[nn].ll, pos, value);
35     }
36     else{
37         if (st[nn].rr == -1) { //init
38             st[nn].rr = st.size();
39             st.pb(Node(mid + 1, st[nn].r));
40         }
41         updatePos(st[nn].rr, pos, value);
42     }
43     update(nn);
44 }
45 int getRange(int nn, int u, int v){
46     if (u > v) return 0;
47     if (nn == -1) return 0;
48     if (st[nn].l >= u and st[nn].r <= v){
49         //return infos here
50     }
51     if (st[nn].l > v or st[nn].r < u) {
52         return 0;
53     }
54     return getSize(st[nn].ll, u, v) +
55         getSize(st[nn].rr, u, v);
56 }
57 };

```

1.4 Cartesian Tree

Complexity: $O(n)$ build via Monotonic Stack

Binary tree combining heap-order on values and BST-order on indices

- **Properties:** Min-Heap on values ($val[\text{parent}] \leq val[\text{child}]$) + In-order yields original array indices ($1..n$). Root is global minimum. Subtree of u corresponds to the maximal subarray $[L_u, R_u]$ where $A[u]$ is the minimum ($size(u) = R_u - L_u + 1$).

- $O(n)$ **Stack Build:** Maintain the right spine in a stack. For $i = 1..n$: pop u with $A[u] > A[i]$; last popped becomes $left[i] = u$; remaining top gets $right[\text{top}] = i$. Push i .

- **Key Applications:**

- RMQ $\iff$ LCA: Range minimum $\min_{l \leq k \leq r} A[k] = A[\text{LCA}(l, r)]$.
- Max Rectangle in Histogram: $\max_u (A[u] \cdot size(u))$.
- D&C to Tree DP: Transforms array D&C splitting at minimum into subtree DP.

1.5 Persistent Segment Tree

Complexity: $O(\log n)$ per query/update

Persistent Segment Tree supporting point updates and historic version queries

```

1 struct Node{ //Use Long long if MLE.
2     int sum;
3     //int num;
4     int leftChild;
5     int rightChild;
6
7     Node(int _sum = 0, int _leftChild = 0, int _rightChild = 0){
8         sum = _sum;
9         leftChild = _leftChild;
10        rightChild = _rightChild;
11    }
12 };
13
14 vector<Node> st;
15 vector<int> version;
16
17 void update(int nn, int l, int r, int pos, int val){
18     if (l == r){
19         st[nn].sum = val;
20         return;
21     }
22     int mid = (l + r) >> 1;
23     if (pos <= mid){
24         st.push_back(st[st[nn].leftChild]);
25         st[nn].leftChild = st.size() - 1;
26         update(st[nn].leftChild, l, mid, pos, val);
27     }
28     else{
29         st.push_back(st[st[nn].rightChild]);
30
31         st[nn].rightChild = st.size() - 1;
32         update(st[nn].rightChild, mid + 1, r, pos, val);
33     }
34
35     st[nn].sum = st[st[nn].leftChild].sum + st[st[nn].rightChild].sum;
36 }
37
38 int get(int nn, int l, int r, int u, int v){
39     if (l >= u and r <= v){
40         return st[nn].sum;
41     }
42     if (l > v or r < u) return 0;
43     int mid = (l + r) >> 1;
44     return get(st[nn].leftChild, l, mid, u, v)
45         + get(st[nn].rightChild, mid + 1, r, u, v);
46 }
47
48 ll getKth(int nn, int l, int r, int k){ //Remember to add variable
        "number" to persistent
49     int leftChild = st[nn].leftChild;
50     int rightChild = st[nn].rightChild;
51     int mid = (l + r) >> 1;
52     if (l == r){
53         return value[l] * min(st[nn].num, k * 1LL);
54     }
55     if (st[leftChild].num < k){
56         return st[leftChild].sum + getKth(rightChild, mid + 1, r, k -
            st[leftChild].num);
57     }
58     else return getKth(leftChild, l, mid, k);
59 }
60
61
62 //Use:
63 st.pb(Node(0, 0, 0));
64 version.pb(0);
65 update(version[0], 1, n, i, a[i]);
66
67 { //Add a copy version of the kth version

```

```

68 int k;
69 cin >> k;
70 k--;
71 version.pb(st.size());
72 st.pb(st[version[k]]);
73 }
74 { //Modify and get Range
75 if (type == 1){ //Update pos
76     update(version[k], 1, n, a, b);
77 }
78 else if (type == 2){ //Get Range
79     cout << get(version[k], 1, n, a, b) << endl;
80 }
81 }

```

1.6 Parallel Binary Search

Complexity: $\mathcal{O}((N + Q) \log(\text{range}) \log N)$

Offline simultaneous binary search over multiple monotonic queries

```

1 void PBS(int l, int r, vector<int>& candi) {
2 if(l + 1 == r || candi.empty()) { //trivial case.
3 for(auto c : candi) ans[c] = 1;
4 return;
5 }
6 int mid = (l + r) >> 1;
7 // do things: events from [l, mid)
8
9 // split candi into two parts
10 vector<int> ok, not_ok;
11 for(auto c : candi) {
12 ull sum = 0;
13 //Query information
14
15 if(target[c] <= sum /* Ok Condition */) ok.push_back(c);
16 else {
17     target[c] -= sum; //Reduce the condition
18     not_ok.push_back(c);
19 }
20 }
21 // undo things: events from [l, mid)
22
23 // continue binary search and free memory
24 PBS(l, mid, ok); vector<int> ().swap(ok);
25 PBS(mid, r, not_ok); vector<int> ().swap(not_ok);
26 }

```

1.7 Wavelet Tree

Complexity: $\mathcal{O}(\log(\text{alphabet}))$ per query

Wavelet Tree for range k -th element, range frequency, and range counting in $[L, R]$

```

1 // Complexity:  $\mathcal{O}(N \log \Sigma)$  build,  $\mathcal{O}(\log \Sigma)$  query,  $\mathcal{O}(N \log \Sigma)$  space
2 // Wavelet Tree: Compact structure for range k-th, range count, and pred/succ
3
4 1. Structure & Index Mapping:
5 - Node represents value range [L, R] with mid = (L + R) / 2.
6 - Sequence at node keeps original relative order:
7   Elements <= mid go to Left child, > mid go to Right child.
8 - b[i] = count of elements <= mid in prefix [0..i] (1-based: b[i] = b[i-1] + (A[i] <= mid)).
9 - Index transformation for range [l, r]:
10   Left child range: [b[l - 1] + 1, b[r]]
11   Right child range: [l - b[l - 1], r - b[r]]

```

```

12
13 2. Core Queries:
14 function kth_smallest(node, l, r, k):
15     if L == R: return L
16     c = b[r] - b[l - 1] // count going left
17     if k <= c:
18         return kth_smallest(Left, b[l - 1] + 1, b[r], k)
19     else:
20         return kth_smallest(Right, l - b[l - 1], r - b[r], k - c)
21
22 function count_le(node, l, r, k): // count elements <= k in [l, r]
23     if l > r or k < L: return 0
24     if R <= k: return r - l + 1
25     c = b[r] - b[l - 1]
26     return count_le(Left, b[l - 1] + 1, b[r], k) +
27         count_le(Right, l - b[l - 1], r - b[r], k)
28
29 function count_in_range(l, r, low, high):
30     return count_le(l, r, high) - count_le(l, r, low - 1)
31
32 3. Implementation Tips:
33 - Coordinate Compression: Compress values to [1, distinct_N] so depth is  $\mathcal{O}(\log N)$ .
34 - Bitvector Optimization: Store b[] as 64-bit integers; use __builtin_popcountll for  $\mathcal{O}(1)$  rank.
35 - Swap Adjacent: Swapping A[i] and A[i+1] only updates 1 bit per level in  $\mathcal{O}(\log \Sigma)$ .

```

1.8 Segment Tree Beat

Complexity: $\mathcal{O}(n \log n)$ amortized

Segment Tree Beats: Range Chmin/Chmax, historical min/max, and range sum queries

```

1 // Complexity:  $\mathcal{O}((N + Q) \log N)$  amortized;  $\mathcal{O}((N + Q) \log^2 N)$  with Range Add
2 // Segment Tree Beats (Ji Driver Segment Tree) - Core Ideas & Principles
3
4 1. Core Concept & Motivation:
5 - Target operations: Range Chmin (A[i] = min(A[i], x)), Range Chmax, Range Add, Range Sum.
6 - Key Insight: A range chmin(x) only modifies elements strictly greater than x.
7   By isolating the strictly maximum elements, we can update them in  $\mathcal{O}(1)$  without
8   affecting any other elements in the segment.
9
10 2. Maintained Node State:
11 - max1: Strictly largest value in node range.
12 - max2: Strictly second largest value (max2 < max1; -INF if all values equal).
13 - cnt_max: Number of occurrences of max1 in node range.
14 - sum: Sum of all elements in node range.
15 - (Symmetrically maintain min1, min2, cnt_min if Range Chmax is also needed).
16 - Lazy tags: lazy_add (additive shift), lazy_max (chmin bound).
17
18 3. The Three Update Conditions for Range Chmin(x):
19 - Break Condition (Outside or No-Op):
20   Node interval is disjoint from query interval, OR x >= max1.
21   Action: Return immediately (no value in subtree can change).
22 - Tag Condition (Direct  $\mathcal{O}(1)$  Update):
23   Node interval is fully inside query interval, AND max2 < x < max1.
24   Action: Only the cnt_max elements equal to max1 decrease to x.

```

```

25     sum -= cnt_max * (max1 - x); max1 = x; set lazy tag;
26     return.
27 - Recursion Condition (Multiple Values Affected):
28   Node interval is fully inside query interval, BUT x <= max2.
29   Action: Both max1 and max2 (and potentially others) change.
30   Push down pending tags, recurse into both children, then pull/merge.
31
32 4. Maintenance Rules (Pull & Push):
33 - Pull (Merge from children L, R):
34   * sum = L.sum + R.sum
35   * max1 = max(L.max1, R.max1)
36   * cnt_max = sum of cnt_max of children having child.max1 == parent.max1.
37   * max2 = largest value among {L.max1, L.max2, R.max1, R.max2} strictly less than max1.
38 - Push Down (Lazy Propagation):
39   * Additive tags apply to all elements (shift sum, max1, max2, min1, min2).
40   * Chmin tag u.max1 only pushes to children with child.max1 > u.max1:
41     child.sum -= child.cnt_max * (child.max1 - u.max1), child.max1 = u.max1.
42
43 5. Complexity & Potential Analysis:
44 - Define potential Phi = total number of distinct values across all nodes in tree.
45 - Each recursion into the 3rd case (x <= max2) forces at least two distinct values
46   to merge into one, strictly decreasing Phi by >= 1.
47 - Chmin/Chmax + Sum: amortized  $\mathcal{O}((N + Q) \log N)$ .
48 - Range Add creates at most  $\mathcal{O}(\log N)$  new distinct values -> amortized  $\mathcal{O}((N + Q) \log^2 N)$ .
49
50 6. Common Extensions:
51 - Range Divide (A[i] = floor(A[i] / d)): Convert to Range Add when
52   max1 - floor(max1 / d) == min1 - floor(min1 / d), otherwise recurse.
53 - Range Modulo (A[i] = A[i] % d): Break if max1 < d; otherwise recurse
54   (each recursion strictly halves values >= d).
55 - Historical Min/Max/Sum: Maintain historical max tag alongside current tag.

```

1.9 Kd Tree

Complexity: $\mathcal{O}(\sqrt{n})$ per query

K-Dimensional Tree for k -NN nearest neighbor and orthogonal range queries in 2D/3D

```

1 struct point{
2 double dim[2];
3 bool operator != (const point &other){
4     return !(dim[0] == other.dim[0] and dim[1] == other.dim[1]);
5 }
6 } pt[MAXN], tmp[MAXN];
7 struct cmp_point{
8 int axis;
9 cmp_point(int x = 0): axis(x) {}
10 bool operator () (const point &a, const point &b){
11     return a.dim[axis] < b.dim[axis];
12 }
13 };
14 struct kd_tree{
15 int axis;
16 double value;
17 point p;

```

```

18 kd_tree *left ,*right;
19 };
20 typedef kd_tree* node_ptr;
21 node_ptr node;
22 double distance_squared(point a, point b){
23 double sum = 0;
24 for(int i = 0; i < 2; i++){
25     sum += (a.dim[i] - b.dim[i]) * (a.dim[i] - b.dim[i]);
26 }
27 return sqrt(sum);
28 }
29 void build(node_ptr &node, int l, int r, int axis){
30 if (l > r){
31     node = NULL;
32     return;
33 }
34 node = new kd_tree();
35 if (l == r){
36     node->p = pt[l];
37     node->left = node->right = NULL;
38     node->axis = axis;
39     return;
40 }
41 int mid = (l + r) >> 1;
42 nth_element(pt + l, pt + mid, pt + 1 + r, cmp_point(axis));
43 node->value = pt[mid].dim[axis]; node->axis = axis; node->p =
    pt[mid];
44 build(node->left, l, mid - 1, axis ^ 1);
45 build(node->right, mid + 1, r, axis ^ 1);
46 }
47 void NearestNeighbour(node_ptr node, point q, double &ans){
48 if (node == NULL) return;
49 if (q != node->p) minimize(ans, distance_squared(node->p, q));
50 if (node->left == NULL and node->right == NULL){
51     return;
52 }
53 int axis = node->axis;
54 double value = node->value;
55 if (q.dim[axis] <= value){
56     NearestNeighbour(node->left, q, ans);
57     if (q.dim[axis] + ans >= value + eps) NearestNeighbour(node->
        right, q, ans);
58 }
59 else{
60     NearestNeighbour(node->right, q, ans);
61     if (q.dim[axis] - ans <= value - eps) NearestNeighbour(node->
        left, q, ans);
62 }
63 }

```

1.10 BIT2D

Complexity: $\mathcal{O}(\log^2 n)$ per query
 2D Binary Indexed Tree (Fenwick) for 2D range sum queries and point updates

```

1 int BIT2D[1003][1003];
2 void update(int x, int y, int val){
3     for(int i = x; i <= n; i += i & (-i)){
4         for(int j = y; j <= m; j += j & (-j)){
5             BIT2D[i][j] += val;
6         }
7     }
8 }
9
10 int get(int x, int y){
11     int res = 0;
12     for(int i = x; i > 0; i -= i & (-i)){

```

```

13         for(int j = y; j > 0; j -= j & (-j)){
14             res += BIT2D[i][j];
15         }
16     }
17     return res;
18 }

```

1.11 Treap

Complexity: $\mathcal{O}(\log n)$ expected per operation
 Randomized Binary Search Tree supporting array split/merge and implicit key operations

```

1 int RAND(){
2 int res = 0;
3 FOR(i, 1, 30){
4     res += (1LL << (rand() % 30));
5 }
6 return res;
7 }
8 struct TreapNode{
9 TreapNode *leftChild, *rightChild;
10 int val = 0, sum = 0, weight;
11 int lazy = 0, sz = 0;
12
13 TreapNode(int _val = 0){
14     val = _val;
15     sum = val;
16     weight = RAND();
17     lazy = 0;
18     sz = 1;
19     leftChild = rightChild = nullptr;
20 }
21 };
22 typedef TreapNode* Treap;
23 int getSum(Treap root){
24 if (root == nullptr) return 0;
25 return root->sum;
26 }
27 int getSize(Treap root){
28 if (root == NULL) return 0;
29 return root->sz;
30 }
31 void update(Treap &root){
32 if (root == NULL) return;
33 root->sum = root->val + getSum(root->leftChild) + getSum(root->
    rightChild);
34 root->sz = 1 + getSize(root->leftChild) + getSize(root->rightChild)
    ;
35 }
36 void fix(Treap root){
37 if (root == NULL) return;
38 if (root->lazy){
39     swap(root->leftChild, root->rightChild);
40     if (root->leftChild != NULL) root->leftChild->lazy ^= 1;
41     if (root->rightChild != NULL) root->rightChild->lazy ^= 1;
42 }
43 root->lazy = 0;
44 }
45 void split(Treap rt, Treap &l, Treap &r, int key, int add){
46
47 if (rt == NULL) {
48     l = r = NULL;
49     return;
50 }
51 fix(rt);
52
53 int imkey = add + getSize(rt->leftChild);

```

```

54 if (imkey <= key){
55     l = rt;
56     split(rt->rightChild, l->rightChild, r, key, imkey + 1);
57     update(rt);
58 }
59 else{
60     r = rt;
61     split(rt->leftChild, l, r->leftChild, key, add);
62     update(rt);
63 }
64 }
65
66 void merge(Treap &rt, Treap l, Treap r){
67 fix(l);
68 fix(r);
69 if (!l or !r){
70     rt = ((!l) ? r : l);
71     return;
72 }
73
74 if (l->weight < r->weight){
75     rt = r;
76     merge(rt->leftChild, l, r->leftChild);
77     update(rt);
78 }
79 else{
80     rt = l;
81     merge(rt->rightChild, l->rightChild, r);
82     update(rt);
83 }
84 }
85
86 void go(Treap rt){
87 if (rt == NULL) return;
88
89 go(rt->leftChild);
90 cout << rt->val << ' ';
91
92 go(rt->rightChild);
93
94 }
95 Treap rt;

```

1.12 Splay Tree / Link - cut Tree

Complexity: $\mathcal{O}(\log n)$ amortized per operation
 Link-Cut Tree supporting dynamic path queries, link, cut, and re-root on forests

```

1 struct Node {
2 int x;
3 Node *l = 0;
4 Node *r = 0;
5 Node *p = 0;
6 bool rev = false;
7 Node() = default;
8 Node(int v) { x = v; }
9 void push() {
10 if (rev) {
11     rev = false;
12     swap(l, r);
13     if (l) l->rev ^= true;
14     if (r) r->rev ^= true;
15 }
16 }
17 bool is_root() { return p == 0 || (p->l != this && this != p->r); }
18 };
19 struct LCT {

```

```

20 vector<Node> a;
21 LCT(int n) {
22     a.resize(n + 1);
23     for (int i = 1; i <= n; ++i) a[i].x = i;
24 }
25 void rot(Node *c) {
26     auto p = c->p;
27     auto g = p->p;
28     if (!p->is_root()) (g->r == p ? g->r : g->l) = c;
29     p->push();
30     c->push();
31     if (p->l == c) { // rtl
32         p->l = c->r;
33         c->r = p;
34         if (p->l) p->l->p = p;
35     } else { // rtr
36         p->r = c->l;
37         c->l = p;
38         if (p->r) p->r->p = p;
39     }
40     p->p = c;
41     c->p = g;
42 }
43 void splay(Node *c) {
44     while (!c->is_root()) {
45         auto p = c->p;
46         auto g = p->p;
47         if (!p->is_root()) rot((g->r == p) == (p->r == c) ? p : c);
48         rot(c);
49     }
50     c->push();
51 }
52 Node *access(int v) {
53     Node *last = 0;
54     Node *c = &a[v];
55     for (Node *p = c; p; p = p->p) {
56         splay(p);
57         p->r = last;
58         last = p;
59     }
60     splay(c);
61     return last;
62 }
63 void make_root(int v) {
64     access(v);
65     auto *c = &a[v];
66     if (c->l) c->l->rev ^= true, c->l = 0;
67 }
68 void link(int u, int v) {
69     make_root(u);
70     Node *c = &a[u];
71     c->p = &a[v];
72 }
73 void cut(int u) {
74     access(u);
75     assert(a[u].l);
76     a[u].l->p = 0;
77     a[u].l = 0;
78 }
79 int lca(int u, int v) {
80     if (u == v) return u;
81     access(u); access(v);
82     if (a[u].p == 0) return -1;
83     splay(&a[u]);
84     return (a[u].p ? a[u].p->x : a[u].x);
85 }
86 };

```

1.13 Farach - Colton - Bender

Complexity: $\mathcal{O}(n)$ build, $\mathcal{O}(1)$ query
 $\langle \mathcal{O}(n), \mathcal{O}(1) \rangle$ LCA via Euler Tour ± 1 RMQ and Four Russians speedup

```

1 // Complexity:  $\mathcal{O}(N)$  preprocessing,  $\mathcal{O}(1)$  query,  $\mathcal{O}(N)$  space
2 // Lowest Common Ancestor (LCA) via  $\pm 1$  RMQ & Four Russians
  Speedup
3
4 1. Reduction (Tree LCA  $\rightarrow \pm 1$  RMQ):
5   - Run Euler Tour DFS: record traversal order E[] and depth D[]
     of length  $M = 2N - 1$ .
6   - first[u] = index of first occurrence of node u in E[].
7   - LCA(u, v) = E[ argmin_{i in [first[u], first[v]]} D[i] ].
8   - Property:  $|D[i] - D[i - 1]| = 1$  for all adjacent steps ( $\pm 1$  RMQ).
9
10 2. Block Partitioning & Sparse Table:
11   - Block size  $B = \max(1, \text{floor}(\log_2(M) / 2))$ , number of blocks  $K$ 
     =  $\text{ceil}(M / B)$ .
12   - Compute minimum index of each block: block_min[0..K-1].
13   - Build Sparse Table over block_min[] in  $\mathcal{O}(K \log K) = \mathcal{O}(N)$  time
     and space.
14
15 3. In-Block Precomputation via Bitmasks (Four Russians):
16   - For block b, bit (j - 1) = 1 if  $D[b*B + j] > D[b*B + j - 1]$ 
     else 0.
17   - Only  $2^{(B - 1)} \leq \sqrt{M}$  unique block profiles exist.
18   - Precompute all-pairs RMQ table of size  $B \times B$  for each unique
     mask in  $\mathcal{O}(B^2)$ .
19   - Total table precomputation:  $\mathcal{O}(2^B * B^2) = \mathcal{O}(\sqrt{N} \log^2 N)$ 
     =  $\mathcal{o}(N)$ .
20
21 4. Query LCA / RMQ in  $\mathcal{O}(1)$ :
22     function query_rmq(L, R):
23         bL = L / B, bR = R / B
24         if bL == bR:
25             return in_block_rmq(bL, L % B, R % B)
26         ans = min_by_depth(in_block_rmq(bL, L % B, B - 1),
27                             in_block_rmq(bR, 0, R % B))
28         if bL + 1 < bR:
29             ans = min_by_depth(ans, sparse_table_query(bL + 1, bR -
30                                                         1))
31         return ans
32
33     function query_lca(u, v):
34         L = min(first[u], first[v]), R = max(first[u], first[v])
35         return E[query_rmq(L, R)]

```

1.14 Trie

Complexity: $\mathcal{O}(\text{length})$ per query
 Binary/Alphabet Prefix Tree for dictionary lookup and XOR max queries

```

1 //Vector Implementation
2 struct TrieNode{
3     int child[2];
4     int sz = 0;
5     TrieNode(){
6         memset(child, -1, sizeof(child));
7         sz = 0;
8     }
9 };
10
11 const int B = 30;
12 struct Trie{
13     vector<TrieNode> T = {TrieNode()};

```

```

14
15 void addNum(int num, int s){
16     int p = 0;
17     for(int i = B; i >= 0; i--){
18         int b = (num >> i)&1;
19         if (T[p].child[b] == -1){
20             T[p].child[b] = T.size();
21             T.pb(TrieNode());
22         }
23         T[p].sz += s;
24         p = T[p].child[b];
25     }
26     T[p].sz += s;
27 }
28
29 bool exist(int num){
30     int p = 0;
31     for(int i = B; i >= 0; i--){
32         int b = (num >> i)&1;
33         if (T[p].child[b] == -1) return false;
34         p = T[p].child[b];
35     }
36     return T[p].sz;
37 }
38
39 int getMinXor(int num){
40     int res = 0;
41     int p = 0;
42     for(int i = B; i >= 0; i--){
43         int b = (num >> i)&1;
44         int c1 = T[p].child[b];
45         int c2 = T[p].child[!b];
46
47         if (c1 != -1 and T[c1].sz > 0) p = c1;
48         else if (c2 != -1 and T[c2].sz > 0) {
49             p = c2;
50             res += (1LL << i);
51         }
52     }
53     return res;
54 }
55 };
56
57 //-----
58 //Pointer Implementation
59 struct Trie
60 {
61     struct Node
62     {
63         Node *child[26];
64         int cnt, is_end;
65         Node()
66         {
67             for (int i = 0; i < 26; i++) child[i] = 0;
68             cnt = is_end = 0;
69         }
70     };
71     Node *root, *cur;
72     Trie()
73     {
74         root = new Node();
75     }
76
77     void add(string s)
78     {
79         cur = root;
80         for (char i : s)
81         {
82             if (cur->child[i - 'a'] == NULL) cur->child[i - 'a'] = new

```

```

    Node();
83     cur = cur->child[i - 'a'];
84     cur->cnt++;
85 }
86 cur->is_end++;
87 }
88
89 bool check(string s)
90 {
91     cur = root;
92     for (char i : s)
93     {
94         if (cur->child[i - 'a'] == NULL) return(0);
95         cur = cur->child[i - 'a'];
96     }
97     return(cur->is_end);
98 }
99
100 int cnt(string s)
101 {
102     cur = root;
103     for (char i : s)
104     {
105         if (cur->child[i - 'a'] == NULL) return(0);
106         cur = cur->child[i - 'a'];
107     }
108     return(cur->cnt);
109 }
110
111 void del(string s)
112 {
113     stack <Node*> st;
114     cur = root;
115     for (int i = s.size() - 1; i >= 0; i--)
116     {
117         st.push(cur);
118         cur = cur->child[s[i] - 'a'];
119         cur->cnt--;
120     }
121     st.push(cur);
122     for (int i = 0; i < s.size(); i++)
123     {
124         if (st.top()->cnt == 0)
125         {
126             delete(st.top());
127             st.pop();
128             st.top()->child[s[i] - 'a'] = NULL;
129         }
130     }
131 }
132 };

```

1.15 Lichao Tree

Complexity: $\mathcal{O}(\log n)$ per query/insertion
Li Chao Segment Tree for online dynamic line insertions and minimum/maximum evaluation

```

1  const int maxn = 2e5;
2
3  point line[4 * maxn];
4
5  void add_line(point nw, int v = 1, int l = 0, int r = maxn) {
6      int m = (l + r) / 2;
7      bool lef = f(nw, l) < f(line[v], l);
8      bool mid = f(nw, m) < f(line[v], m);
9      if(mid) {
10         swap(line[v], nw);

```

```

11     }
12     if(r - l == 1) {
13         return;
14     } else if(lef != mid) {
15         add_line(nw, 2 * v, l, m);
16     } else {
17         add_line(nw, 2 * v + 1, m, r);
18     }
19 }

```

1.16 Line Container

Complexity: $\mathcal{O}(\log n)$ amortized per operation
Convex Hull of dynamic lines with arbitrary slopes queried at arbitrary x

```

1  ///// line container start (max query) /////
2  struct Line {
3      mutable ll k, m, p;
4      bool operator<(const Line& o) const { return k < o.k; }
5      bool operator<(ll x) const { return p < x; }
6  }; // (for doubles, use inf = 1/.0, div(a,b) = a/b)
7  struct LineContainer : multiset<Line, less<>> {
8      static const ll inf = LLONG_MAX; // div: floor
9      ll div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }
10     bool isect(iterator x, iterator y) {
11         if (y == end()) return x->p = inf, 0;
12         if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
13         else x->p = div(y->m - x->m, x->k - y->k);
14         return x->p >= y->p;
15     }
16     void add(ll k, ll m) {
17         auto z = insert({k, m, 0}), y = z++, x = y;
18         while (isect(y, z)) z = erase(z);
19         if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
20         while ((y = x) != begin() && (--x)->p >= y->p) isect(x,
21             erase(y));
22         ll query(ll x) { assert(!empty());
23             auto l = *lower_bound(x); return l.k * x + l.m; }
24 };

```

1.17 Arpa Trick

Complexity: $\mathcal{O}(n\alpha(n))$
Fast offline range maximum/minimum query via DSU

```

1  Offline RMQ in  $\mathcal{O}(N \cdot \alpha(N))$  using DSU and Monotonic Stack.
2
3  Idea:
4  1. Group queries by their right endpoint  $R$ .
5  2. Iterate  $R$  from  $1$  to  $N$ , maintaining a monotonic stack of
   indices of strictly decreasing elements.
6  3. When the current element  $a[R]$  pops an index  $u$  from the stack
   (because  $a[u] \leq a[R]$ ), we merge  $u$  into  $R$  in a DSU by
   setting  $parent[u] = R$ .
7  4. After updating the stack and DSU for  $R$ , to answer a query  $[L, R]$ :
8     - The index of the maximum element in  $[L, R]$  is exactly '
   find_root(L)'.
9
10 Why it works: 'find_root(L)' follows the chain of "next greater
   elements" and stops at the largest element  $\leq R$  (since
   elements  $\geq R$  haven't been processed yet).

```

1.18 Blocky Data Structure

Complexity: $\mathcal{O}(n\sqrt{n})$
Square root decomposition: Block Array and Hilbert-ordered Mo's algorithm

```

1
2 //Sqrt Decomposition DS
3 //Query:  $\mathcal{O}(1)$ 
4 //Update:  $\mathcal{O}(\sqrt{n})$ 
5
6
7 struct BlockyArray{
8     vector<int> arr, lazy; //zero based
9     BlockyArray(int n = 0){
10         arr.resize(n + 9, 0);
11         lazy.resize(n + 9, 0);
12     }
13     void updateUp(int pos, int val){
14         pos++;
15         while(pos < arr.size() and pos % block != 0) arr[pos++] +=
16             val;
17         for(int i = pos; i < arr.size(); i += block) lazy[i /
18             block] += val;
19     }
20     void updateDown(int pos, int val){ //one based
21         //cap nhất nhưng gia tri nho hon
22         pos
23         pos--;
24         while(pos >= 0 and pos % block != block - 1) arr[pos--] +=
25             val;
26         for(int i = pos; i >= 0; i -= block) lazy[i / block] +=
27             val;
28     }
29     int get(int pos){
30         return arr[pos] + lazy[pos / block];
31     }
32 };
33
34 //MO - Hilbert
35 inline int64_t hilbertOrder(int x, int y, int pow, int rotate) {
36     if (pow == 0) { return 0; }
37     int hpow = 1 << (pow-1);
38     int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 : 2);
39     seg = (seg + rotate) & 3;
40     const int rotateDelta[4] = {3, 0, 0, 1};
41     int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
42     int nrot = (rotate + rotateDelta[seg]) & 3;
43     int64_t subSquareSize = int64_t(1) << (2*pow - 2);
44     int64_t ans = seg * subSquareSize;
45     int64_t add = hilbertOrder(nx, ny, pow-1, nrot);
46     ans += ((seg == 1 || seg == 2) ? add : (subSquareSize - add - 1));
47     return ans;
48 }
49
50 struct Query{
51     int l, r, id, ord;
52     Query(int _l, int _r, int _id){
53         l = _l; r = _r; id = _id;
54         ord = hilbertOrder(l, r, 17, 0);
55     }
56     bool operator < (const Query &other) const{ return ord < other.
57         ord;}
58 };
59
60 vector<Query> query;
61 void pseudo_mo(int lo, int hi, int id){
62     while(r < hi){

```

```

56     sweeplineM0[l - 1].pb(Segment(r + 1, hi, -1, id));
57     r = hi;
58 }
59 while(l > lo){
60     sweeplineM0[r].pb(Segment(lo, l - 1, 1, id));
61     l = lo;
62 }
63 while(r > hi){
64     sweeplineM0[l - 1].pb(Segment(hi + 1, r, 1, id));
65     r = hi;
66 }
67 while(l < lo){
68     sweeplineM0[r].pb(Segment(l, lo - 1, -1, id));
69     l = lo;
70 }
71 }

```

1.19 Ultimate Hashmap

Complexity: $\mathcal{O}(1)$ expected per lookup
Fast custom hash map with custom splitmix64 hash function

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3 gp_hash_table<int, int> table;
4
5 const int RANDOM = chrono::high_resolution_clock::now().
    time_since_epoch().count();
6 struct chash {
7     int operator()(int x) const { return x ^ RANDOM; }
8 };
9 gp_hash_table<key, int, chash> table;

```

1.20 Erasable Priority Queue

Complexity: $\mathcal{O}(\log n)$ per operation
Max/Min Priority Queue with lazy removal of arbitrary elements

```

1 template<class T=int, class O=less<T>>
2 struct pq_set {
3     priority_queue<T, vector<T>, O> q, del;
4     const T& top() const { return q.top(); }
5     int size() const { return (int)q.size() - del.size(); }
6     bool empty() const { return !size(); }
7     void insert(const T x) { q.push(x); flush(); }
8     void pop() { q.pop(); flush(); }
9     void erase(const T x) { del.push(x); flush(); }
10    void flush() { while(del.size() && q.top() == del.top())
11        q.pop(), del.pop(); }
12 };

```

1.21 Expandable Data Structure

Complexity: $\mathcal{O}(A \log n)$ amortized insertion, for static structure buildable in $\mathcal{O}(A)$

Binary doubling technique: converts static data structures (e.g. Aho, CHT) to support online insertions

```

1 void Insert(){ //Insert something here.
2     for(int i = 0; i < 20; i++){
3         if (block[i].size() > (1 << i)){
4             //Push the element onto the next set
5         }
6     }
7     //rebuild the data structure
8     break;

```

```

9 }
10 }
11 }
12 void query(){
13     for(int i = 0; i < 20; i++){
14         //query on each block
15     }
16 }

```

1.22 Interval Set

Complexity: $\mathcal{O}(\log n)$ amortized per operation
Disjoint interval set (ODT style): maintains non-overlapping intervals, range assignment, and adjacent merging based on predicate.

```

1 struct Node {
2     int l, r; char c;
3     bool operator<(const Node& o) const { return l < o.l; }
4 };
5 struct IntervalSet : set<Node> {
6     bool can_merge(char a, char b) const { return a == b; }
7
8     auto assign(int l, int r, char c) {
9         vector<Node> rem, add;
10
11         // 1. Find first interval intersecting with [l, r]
12         auto it = upper_bound({l, (int)2e9, 0});
13         if (it != begin() && prev(it)->r >= l) --it;
14
15         // 2. Erase intersecting intervals & preserve leftovers at
16         // both ends
17         while (it != end() && it->l <= r) {
18             auto [L, R, C] = *it;
19             rem.push_back(*it);
20             it = erase(it);
21             if (L < l) add.push_back(*insert({l, l - 1, C}).first);
22             if (R > r) add.push_back(*insert({r + 1, R, C}).first);
23         }
24
25         // 3. Merge with neighbors if can_merge holds
26         it = lower_bound({l, 0, 0});
27         if (it != begin() && prev(it)->r == l - 1 && can_merge(prev(it)->c, c)) {
28             l = prev(it)->l; rem.push_back(*prev(it)); erase(prev(it));
29         }
30         if (it != end() && it->l == r + 1 && can_merge(it->c, c)) {
31             r = it->r; rem.push_back(*it); erase(it);
32         }
33
34         // 4. Insert new interval
35         add.push_back(*insert({l, r, c}).first);
36         return pair{rem, add};
37 };

```

2 Graph

2.1 Minimum Cost Flow

Complexity: $\mathcal{O}(F \cdot m \log n)$
Min-cost K Flow: Transfer k packages through graph with capacities and costs

```

1 // Complexity:  $\mathcal{O}(F \cdot M \log N)$  using SPFA / Dijkstra with node
    potentials
2 // Finds min cost to send F units of flow from source s to sink t

```

```

3
4 struct Edge:
5     to, rev, cap, flow = 0, cost
6
7 function min_cost_flow(s, t, max_flow_target):
8     pot[N] = 0, total_flow = 0, total_cost = 0
9     // Compute initial potential with SPFA if negative costs exist
10
11     while total_flow < max_flow_target:
12         priority_queue pq // (dist, u)
13         dist[0..N-1] = INF, parent_edge[0..N-1] = -1
14         dist[s] = 0, pq.push({0, s})
15
16         while not pq.empty():
17             d, u = pq.pop()
18             if d > dist[u]: continue
19             for e in adj[u]:
20                 if e.cap - e.flow > 0:
21                     reduced_cost = e.cost + pot[u] - pot[e.to]
22                     if dist[e.to] > dist[u] + reduced_cost:
23                         dist[e.to] = dist[u] + reduced_cost
24                         parent_edge[e.to] = e
25                         pq.push({dist[e.to], e.to})
26
27             if dist[t] == INF: break // No more augmenting path
28             for i = 0 to N - 1: pot[i] += dist[i]
29
30         push_flow = max_flow_target - total_flow
31         v = t
32         while v != s:
33             e = parent_edge[v]
34             push_flow = min(push_flow, e.cap - e.flow)
35             v = adj[e.to][e.rev].to
36
37         v = t
38         while v != s:
39             e = parent_edge[v]
40             e.flow += push_flow
41             adj[e.to][e.rev].flow -= push_flow
42             total_cost += push_flow * e.cost
43             v = adj[e.to][e.rev].to
44         total_flow += push_flow
45
46     return (total_flow, total_cost)

```

2.2 MinCut - MaxFlow

Complexity: $\mathcal{O}(EV^2)$

Dinic: Max number of paths / vertex-disjoint paths from source to sink.

• **Grid 2-Coloring:** Partition cells by $(r + c) \bmod 2 \implies$ Bipartite graph. Used for domino tiling, non-attacking pieces, laser grid beam shooting.

• **Vertex Splitting** ($u_{in} \rightarrow u_{out}$): Restricts vertex capacity or forces vertex visit (capacity c_u , cost w_u).

• **Max-Weight Closure (Project Selection):**

– Rule: If choosing item u , must also choose item v (directed edge $u \rightarrow v$ with $\text{cap} = \infty$).

– For positive weight $W_u > 0$: Add edge $S \rightarrow u$ with $\text{cap} = W_u$.

– For negative weight $W_v < 0$: Add edge $v \rightarrow T$ with $\text{cap} = -W_v$.

– Max Profit = $\sum_{W_u > 0} W_u - \text{MinCut}(S, T)$.

• **Two-Choice Binary Partitioning:** Each item i assigned to set S (cost A_i) or T (cost B_i). If $i \in S, j \in T$, penalty $C_{i,j}$. Add $S \rightarrow i(B_i), i \rightarrow T(A_i), i \rightarrow j(C_{i,j})$. Total cost = Min-Cut.

- **Circulation with Demands** $[L_e, R_e]$: Replace edge $e = (u, v)$ with cap $R_e - L_e$. Let $d(u) = \sum_{in} L - \sum_{out} L$. Add super-source S^* , super-sink T^* . For $d(u) > 0$, add $S^* \rightarrow u$ with cap $d(u)$; for $d(u) < 0$, add $u \rightarrow T^*$ with cap $-d(u)$. Feasible $\iff$ all S^* edges saturated.

```

1 // Complexity:  $O(V^2 * E)$  general,  $O(E * \sqrt{V})$  for unit networks
  / bipartite matching
2 // Dinic's Algorithm: Repeated BFS level graph construction and DFS
  blocking flow

3
4 struct Edge:
5     to, rev, cap, flow = 0
6
7 int level[N], ptr[N]

8
9 function bfs_level(s, t):
10     fill(level, -1); level[s] = 0
11     queue Q; Q.push(s)
12     while not Q.empty():
13         u = Q.pop()
14         for e in adj[u]:
15             if e.cap - e.flow > 0 and level[e.to] == -1:
16                 level[e.to] = level[u] + 1
17                 Q.push(e.to)
18     return level[t] != -1

19
20 function dfs_blocking(u, t, pushed):
21     if pushed == 0 or u == t: return pushed
22     for cid = ptr[u] to len(adj[u]) - 1:
23         ptr[u] = cid; e = adj[u][cid]
24         if level[e.to] == level[u] + 1 and e.cap - e.flow > 0:
25             tr = dfs_blocking(e.to, t, min(pushed, e.cap - e.flow))
26             if tr == 0: continue
27             e.flow += tr; adj[e.to][e.rev].flow -= tr
28             return tr
29     return 0

30
31 function dinic_max_flow(s, t):
32     flow = 0
33     while bfs_level(s, t):
34         fill(ptr, 0)
35         while pushed = dfs_blocking(s, t, INF):
36             flow += pushed
37     return flow

```

2.3 Bipartite Matching

Complexity: $O(E\sqrt{V})$

Hopcroft-Karp: Node Cover, Node-disjoint path cover, General path cover, Antichain (Dilworth).

- **König's Theorem:** Max Matching = Min Vertex Cover.
- **Max Independent Set:** $|V| - \text{Min Vertex Cover} = |V| - \text{Max Matching}$.
- **Min Edge Cover** (no isolated vertices): $|V| - \text{Max Matching}$.
- **Min Path Cover on DAG:**

- *Vertex-disjoint:* Split $u \rightarrow u_{out}, u_{in}$; build bipartite matching $u_{out} \rightarrow v_{in}$ for edge $u \rightarrow v$. Result: $|V| - \text{Max Matching}$.
- *General (paths can share vertices):* Compute transitive closure (Floyd-Warshall / reachability), then solve vertex-disjoint path cover.

- **Dilworth's Theorem:** In any finite poset (DAG reachability), the length of the **Maximum Antichain** (largest subset of mutually unreachable vertices) equals the **Minimum Path Cover**.

- **Hall's Marriage Theorem:** Bipartite (X, Y) has a matching saturating $X \iff \forall S \subseteq X, |N(S)| \geq |S|$.

- **Max Matching Size via Hall:** $|X| - \max_{S \subseteq X} (|S| - |N(S)|)$.

```

1 // Complexity:  $O(E * \sqrt{V})$ 
2 // Maximum Bipartite Matching on Left set L [1..n] and Right set R
  [1..m]

3
4 int pair_u[MAXN], pair_v[MAXM], dist[MAXN]

5
6 function bfs_hopcroft():
7     queue Q
8     for u = 1 to n:
9         if pair_u[u] == 0: dist[u] = 0; Q.push(u)
10        else: dist[u] = INF
11    dist[0] = INF
12    while not Q.empty():
13        u = Q.pop()
14        if dist[u] < dist[0]:
15            for v in adj[u]:
16                if dist[pair_v[v]] == INF:
17                    dist[pair_v[v]] = dist[u] + 1
18                    Q.push(pair_v[v])
19    return dist[0] != INF

20
21 function dfs_hopcroft(u):
22     if u != 0:
23         for v in adj[u]:
24             if dist[pair_v[v]] == dist[u] + 1 and dfs_hopcroft(
                pair_v[v]):
25                 pair_v[v] = u; pair_u[u] = v
26                 return True
27     dist[u] = INF
28     return False

29
30
31 function max_matching():
32     fill(pair_u, 0); fill(pair_v, 0); matching = 0
33     while bfs_hopcroft():
34         for u = 1 to n:
35             if pair_u[u] == 0 and dfs_hopcroft(u):
36                 matching += 1
37     return matching

```

2.4 Hungarian Matching

Complexity: $O(V^3)$

Min-cost perfect assignment on bipartite graph

```

1 // Complexity:  $O(N^3)$  time,  $O(N^2)$  space
2 // Minimum weight perfect matching on 1-indexed N x M bipartite
  graph (N <= M)

3
4 function hungarian(cost[1..N][1..M]):
5     u[0..N] = 0, v[0..M] = 0, p[0..M] = 0, way[0..M] = 0
6     for i = 1 to N:
7         p[0] = i, j0 = 0
8         minv[0..M] = INF, used[0..M] = False
9         repeat:
10             used[j0] = True
11             i0 = p[j0], delta = INF, j1 = 0
12             for j = 1 to M:
13                 if not used[j]:
14                     cur = cost[i0][j] - u[i0] - v[j]
15                     if cur < minv[j]: minv[j] = cur; way[j] = j0
16                     if minv[j] < delta: delta = minv[j]; j1 = j
17             for j = 0 to M:
18                 if used[j]: u[p[j]] += delta; v[j] -= delta
19                 else: minv[j] -= delta
20             j0 = j1

```

```

21         until p[j0] == 0
22         repeat:
23             j1 = way[j0]; p[j0] = p[j1]; j0 = j1
24         until j0 == 0
25         // Result: matched left node of right node j is p[j], min cost
        is -v[0]
26     return -v[0]

```

2.5 General Matching

Complexity: $O(V^3)$

Edmonds Blossom: Maximum Matching on general graphs.

- **Tutte Matrix & Rabin-Vazirani** ($O(V^3)$): For $(u, v) \in E$ ($u < v$), set $T_{u,v} = r \in [1, p-1]$, $T_{v,u} = -r \pmod{p}$. Size = $\frac{1}{2} \text{rank}(T)$ (Gauss mod p).

– *Rabin-Vazirani Reconstruction:* Compute $A = T^{-1} \pmod{p}$ (on $2K \times 2K$ basis). Edge (u, v) is in matching $\iff A_{u,v} \neq 0$. Pick (u, v) with $A_{u,v} \neq 0$, match them, then update A in $O(V^2)$ via Schur complement: $A'_{x,y} = A_{x,y} + \frac{A_{x,u}A_{y,v} - A_{x,v}A_{y,u}}{A_{u,v}} \pmod{p}$.

Repeat for all K pairs $\implies O(V^3)$ total.

- **Max b -Matching Reduction** ($\deg(i) \leq a_i$): Đỉnh $i \rightarrow a_i$ slot ($a_i \leftarrow \min(a_i, \deg(i))$). Cạnh $e = (u, v) \rightarrow L_e, R_e$, nối $L_e - R_e$, $L_e - \text{slot}(u)$, $R_e - \text{slot}(v)$. Bỏ e : match $L_e - R_e$ (+1). Chọn e : match L_e, R_e với slot (+2, gain +1). Aux $|V'| \leq 4M \implies$ Blossom.

```

1 // TUTTE MATRIX & RABIN-VAZIRANI MATCHING -  $O(V^3)$ 
2 // -----
3 // p is a large prime (1e9 + 7). All operations mod p.
4
5 function build_tutte_matrix(G):
6     T = zeros(N, N)
7     for each edge (u, v) in G with u < v:
8         r = random(1, p - 1)
9         T[u][v] = r, T[v][u] = p - r
10    return T
11
12 // Symmetric Gaussian Elimination -> Basis S (|S| = 2K)
13 function find_basis(T):
14     M = copy(T), S = empty_list
15     for i = 0 to N - 1:
16         if i in S: continue
17         find pivot j > i (j not in S and M[i][j] != 0)
18         if not found: continue
19         add i, j to S
20         inv = (M[i][j])^(-1) mod p
21         for r = 0 to N - 1 not in S:
22             for c = 0 to N - 1 not in S:
23                 delta = (M[i][c]*M[r][j] - M[j][c]*M[r][i])
24                 M[r][c] = (M[r][c] - delta * inv) mod p
25     return S // |S| = 2K, Max Matching Size = K
26
27 // Reconstruct K matched pairs from full-rank submatrix
28 function rabin_vazirani(G, T, S):
29     A = invert_matrix(T[S][S]) //  $O(|S|^3)$  Gauss-Jordan
30     matching = []
31     while S has unmatched vertices:
32         pick unmatched u in S
33         find unmatched v in S (adj(u, v) and A[u][v] != 0)
34         matching.append((u, v))
35         // Schur complement update in  $O(|S|^2)$ 
36         inv = (A[u][v])^(-1) mod p
37         for each unmatched x in S:
38             for each unmatched y in S:

```

```

39         delta = A[x][u]*A[y][v] - A[x][v]*A[y][u]
40         A[x][y] = (A[x][y] + delta * inv) mod p
41     return matching

```

2.6 Biconnected Component

Complexity: $\mathcal{O}(V + E)$

BCC: Cut vertices, bridges, and 2-connected components

```

1 // Complexity:  $\mathcal{O}(V + E)$  time
2 // Finds Bridges, Articulation Points, and 2-Connected Components (
  Biconnected Components)
3
4 int num[N], low[N], timer = 0
5 stack<pair<int, int>> edge_stack
6
7 function dfs_bcc(u, p):
8     num[u] = low[u] = ++timer
9     children = 0
10    for v in adj[u]:
11        if v == p: continue
12        if num[v]:
13            low[u] = min(low[u], num[v])
14            if num[v] < num[u]: edge_stack.push({u, v})
15        else:
16            children += 1
17            edge_stack.push({u, v})
18            dfs_bcc(v, u)
19            low[u] = min(low[u], low[v])
20
21    // Bridge condition: low[v] > num[u]
22    if low[v] > num[u]: mark_bridge(u, v)
23
24    // Articulation point condition:
25    if (p == -1 and children > 1) or (p != -1 and low[v] >=
  num[u]):
26        mark_articulation_point(u)
27
28    // Extract Biconnected Component:
29    if low[v] >= num[u]:
30        component = []
31        while True:
32            edge = edge_stack.pop()
33            component.append(edge)
34            if edge == {u, v}: break
35        process_bcc(component)

```

2.7 Functional Cycle

Complexity: $\mathcal{O}(\text{dist} + \text{len})$

Floyd Tortoise and Hare: Find cycle in functional graph $f(x)$

```

1 // Complexity:  $\mathcal{O}(k)$  time where k is distance + cycle length,  $\mathcal{O}(1)$ 
  space
2 // Floyd's Tortoise and Hare for functional graph (out-degree of
  each node is 1: f(x))
3
4 function find_cycle(x0, f):
5     tortoise = f(x0), hare = f(f(x0))
6     while tortoise != hare:
7         tortoise = f(tortoise)
8         hare = f(f(hare))
9
10    // Find entry point of cycle (mu)
11    mu = 0, tortoise = x0
12    while tortoise != hare:
13        tortoise = f(tortoise)

```

```

14     hare = f(hare)
15     mu += 1
16
17    // Find cycle length (lambda)
18    lam = 1, hare = f(tortoise)
19    while tortoise != hare:
20        hare = f(hare)
21        lam += 1
22
23    return (entry_node = tortoise, start_dist = mu, cycle_len = lam
  )

```

2.8 Strongly Connected Components

Complexity: $\mathcal{O}(V + E)$

Tarjan SCC on directed graphs

```

1 // Complexity:  $\mathcal{O}(V + E)$  time,  $\mathcal{O}(V)$  space
2 // Tarjan's Strongly Connected Components algorithm for directed
  graphs
3
4 int dfn[N], low[N], scc_id[N], timer = 0, scc_cnt = 0
5 stack<int> st; bool in_st[N]
6
7 function dfs_scc(u):
8     dfn[u] = low[u] = ++timer
9     st.push(u); in_st[u] = True
10    for v in adj[u]:
11        if not dfn[v]:
12            dfs_scc(v)
13            low[u] = min(low[u], low[v])
14        else if in_st[v]:
15            low[u] = min(low[u], dfn[v])
16
17    if low[u] == dfn[u]:
18        scc_cnt += 1
19        repeat:
20            v = st.pop(); in_st[v] = False
21            scc_id[v] = scc_cnt
22        until v == u
23
24 function find_all_sccs():
25     fill(dfn, 0); fill(in_st, False)
26     timer = 0; scc_cnt = 0
27     for u = 1 to N:
28         if not dfn[u]: dfs_scc(u)
29     return scc_id

```

2.9 Offline Deletion

Complexity: $\mathcal{O}(T(n) \log n)$

DSU Rollback with segment tree divide & conquer

```

1 // Complexity:  $\mathcal{O}(\log N)$  per union / rollback (No path compression,
  rank by size)
2 // Supports offline dynamic connectivity & edge deletions via
  segment tree divide & conquer
3
4 struct Operation:
5     u, v, united (bool)
6
7 stack<Operation> history
8 int parent[N], sz[N], components_cnt = N
9
10 function find(u):
11     while u != parent[u]: u = parent[u]
12     return u

```

```

13
14 function unite(u, v):
15     u = find(u); v = find(v)
16     if u == v:
17         history.push({u, v, False})
18         return False
19     if sz[u] < sz[v]: swap(u, v)
20     parent[v] = u; sz[u] += sz[v]; components_cnt -= 1
21     history.push({u, v, True})
22     return True
23
24 function rollback():
25     op = history.pop()
26     if op.united:
27         sz[op.u] -= sz[op.v]
28         parent[op.v] = op.v
29         components_cnt += 1

```

2.10 2-SAT

Complexity: $\mathcal{O}(n + m)$

Boolean 2-Satisfiability via implication graph and SCC.

- **At least one is true** ($X \vee Y$): Add edges $\neg X \rightarrow Y$ and $\neg Y \rightarrow X$.
- **If X then Y** ($X \implies Y$): Add edges $X \rightarrow Y$ and $\neg Y \rightarrow \neg X$.
- **Exactly one is true** ($X \text{ XOR } Y = 1$): $(X \vee Y) \wedge (\neg X \vee \neg Y)$. Edges: $X \leftrightarrow \neg Y$ and $\neg X \leftrightarrow Y$.
- **X and Y must be the same** ($X \text{ XOR } Y = 0$): $(X \vee \neg Y) \wedge (\neg X \vee Y)$. Edges: $X \leftrightarrow Y$ and $\neg X \leftrightarrow \neg Y$.
- **X MUST be true / MUST be false**: Add edge $\neg X \rightarrow X$ (forces true), or $X \rightarrow \neg X$ (forces false).
- **At most one of $\{X_1, X_2, \dots, X_k\}$ is true**: Add prefix variables $P_i = P_{i-1} \vee X_i$. Edges: $X_i \rightarrow P_i$, $P_{i-1} \rightarrow P_i$, and $X_i \rightarrow \neg P_{i-1}$ (if X_i is true, previous must all be false).

```

1 // Complexity:  $\mathcal{O}(N + M)$  time
2 // 2-SAT: Variable xi is mapped to  $2*i$  (True) and  $2*i + 1$  (False).
  Negation of u is  $u \sim 1$ .
3
4 function add_clause_or(u, v):
5     // (u OR v) <=> (NOT u => v) AND (NOT v => u)
6     adj[u ~ 1].append(v)
7     adj[v ~ 1].append(u)
8
9 function solve_2sat(num_vars):
10    find_all_sccs() // on  $2 * \text{num\_vars}$  nodes
11    assignment[1..num_vars]
12    for i = 1 to num_vars:
13        if scc_id[2 * i] == scc_id[2 * i + 1]:
14            return (satisfiable = False, {}) // Contradiction
15        // In topological order of DAG of SCCs, later SCC gets
  chosen:
16        assignment[i] = (scc_id[2 * i] < scc_id[2 * i + 1])
17    return (satisfiable = True, assignment)

```

2.11 Floyd

Complexity: $\mathcal{O}(n^3)$

All-pair Shortest-path / transitive closure / min-max path

```

1 // Complexity:  $\mathcal{O}(V^3)$  time,  $\mathcal{O}(V^2)$  space
2 // All-Pairs Shortest Path and Transitive Closure / Min-Max Path
3
4 function floyd_warshall(dist[N][N]):
5     // dist[i][j] initialized to edge weight or INF, dist[i][i] = 0
6     for k = 1 to N:

```

```

7     for i = 1 to N:
8         for j = 1 to N:
9             if dist[i][k] < INF and dist[k][j] < INF:
10                dist[i][j] = min(dist[i][j], dist[i][k] + dist[
11
12 // Negative cycle detection: if dist[i][i] < 0 for any i

```

2.12 Eulerian Path

Complexity: $\mathcal{O}(m)$

Hierholzer algorithm: Find Eulerian Path/Circuit.

- **Words/Strings Chaining (De Bruijn / Dominoes):** Chaining words where the end of one matches the start of the next. Model *states* (e.g., prefix/suffix of length $L - 1$, or domino halves) as vertices, and the *items* themselves as directed edges. Eulerian path = sequence covering all items exactly once.

- **Assigning Directions to Undirected Edges:** Goal is to orient edges to balance in-degrees and out-degrees (e.g., $|in - out| \leq 1$). Add dummy edges (or connect odd degree vertices to a dummy node) to make all degrees even, find an Eulerian circuit, and orient edges according to the traversal direction.

- **Edge-Disjoint Cycle Decomposition:** Decompose graph into edge-disjoint cycles. Possible $\iff$ graph is connected and all vertices have even degrees. Finding an Eulerian circuit directly yields the cycles.

- **Bipartite Grid Traversal (Row-Col Graphs):** Matrix problems where you alternate moving along rows and columns. Create a bipartite graph of Rows and Columns. A cell (r, c) is an edge between row r and column c . An Eulerian path traverses cells alternating row-col-row.

```

1 // Complexity:  $\mathcal{O}(V + E)$  time and space
2 // Hierholzer's algorithm for finding Eulerian Path / Circuit
3
4 function find_eulerian_path(start_node):
5     path = [], stack st
6     st.push(start_node)
7
8     while not st.empty():
9         u = st.top()
10        if out_degree[u] > 0:
11            // Take an unused edge (u, v), mark edge used
12            v = get_next_unused_edge(u)
13            st.push(v)
14        else:
15            path.append(u)
16            st.pop()
17
18     return reverse(path)

```

2.13 Vizing Theorem

Complexity: $\mathcal{O}(n^2)$

Color graph edges with at most $(\Delta + 1)$ colors

```

1 // Complexity:  $\mathcal{O}(V * E)$  or  $\mathcal{O}(V^2)$  time
2 // Colors the edges of a simple graph using at most (Delta + 1)
   colors (Vizing's theorem)
3
4 function color_edge(u, v0):
5     // Construct fan F = [v0, v1, ..., vk] around u
6     fan = [v0]
7     while can_extend_fan(fan):
8         extend_fan(fan)
9     c = first_color_free_at(u)

```

```

10    d = first_color_free_at(last_fan_node)
11    invert_kempe_chain(u, c, d)
12    shift_fan_colors(fan)
13    assign_color(u, v0, d)
14
15 function vizing_edge_coloring():
16     max_deg = max_degree(G)
17     for (u, v) in edges:
18         color_edge(u, v)
19     return edge_colors

```

2.14 Bipartite Online Checking

Complexity: $\mathcal{O}(\alpha(n))$

Online bipartite checking upon adding edges via parity DSU

```

1 // Complexity:  $\mathcal{O}(\alpha(N))$  per edge addition
2 // Maintain 2-coloring and bipartite validity online using parity-
   weighted DSU
3
4 int parent[N], parity[N], is_bipartite = True
5
6 function find_root(u):
7     if parent[u] == u: return {root: u, par: 0}
8     res = find_root(parent[u])
9     parent[u] = res.root
10    parity[u] = parity[u] ^ res.par
11    return {root: parent[u], par: parity[u]}
12
13 function add_edge(u, v):
14     ru = find_root(u), rv = find_root(v)
15     if ru.root == rv.root:
16         if ru.par == rv.par: // Same color connected by an edge =>
           Odd cycle!
17             is_bipartite = False
18         else:
19             parent[ru.root] = rv.root
20             parity[ru.root] = ru.par ^ rv.par ^ 1

```

2.15 Gomory-Hu Tree

Complexity: $\mathcal{O}(n \cdot \text{MaxFlow})$

Equivalent cut tree for all-pairs max flow / min cut

```

1 // Complexity:  $\mathcal{O}(V * \text{MaxFlow})$  time
2 // Builds equivalent tree where min capacity on tree path between u
   and v equals Min-Cut in G
3
4 function build_gomory_hu_tree(vertices V):
5     parent[1..N] = 1, weight[1..N] = INF
6     for u = 2 to N:
7         p = parent[u]
8         // Compute s-t min cut between u and p in original graph
9         cut_val, reachable_nodes = dinic_min_cut(u, p)
10        weight[u] = cut_val
11
12        for v = u + 1 to N:
13            if v in reachable_nodes and parent[v] == p:
14                parent[v] = u
15
16 // Tree edges are (u, parent[u]) with weight[u] for all u in
   [2..N]

```

2.16 Critical (Dominator Tree)

Complexity: $\mathcal{O}(m \log n)$

Lengauer-Tarjan: Immediate dominators from source s

```

1 // Complexity:  $\mathcal{O}(M \log N)$  or  $\mathcal{O}(M * \alpha(N))$  time (Lengauer-Tarjan
   algorithm)
2 // Immediate Dominator idom[u]: vertex d such that every path from
   source s to u must pass through d
3
4 function lengauer_tarjan(source s):
5     dfs_order(s) // compute dfn[u], parent[u], sdom[u]
6     for i = N down to 2:
7         u = vertex_at_dfn[i]
8         for v in pred[u]:
9             sdom[u] = min(sdom[u], sdom[eval(v)])
10        bucket[vertex_at_dfn[sdom[u]]].append(u)
11        link(parent[u], u)
12        for v in bucket[parent[u]]:
13            u_min = eval(v)
14            idom[v] = parent[u] if sdom[u_min] == sdom[v] else
           u_min
15        bucket[parent[u]].clear()
16
17     for i = 2 to N:
18         u = vertex_at_dfn[i]
19         if idom[u] != vertex_at_dfn[sdom[u]]:
20             idom[u] = idom[idom[u]]

```

2.17 Prufer

Complexity: $\mathcal{O}(n \log n)$

Encode and decode labeled trees of size n

```

1 // Complexity:  $\mathcal{O}(N \log N)$  with priority queue or  $\mathcal{O}(N)$  linear time
2 // Prufer Code: Bijective encoding between labeled trees of size N
   and sequences of length N - 2
3
4
5 function encode_prufer(tree, N):
6     deg = [len(adj[i]) for i in 1..N]
7     leaves = min_priority_queue([i for i in 1..N if deg[i] == 1])
8     code = []
9     for step = 1 to N - 2:
10        leaf = leaves.pop()
11        neighbor = get_only_neighbor(leaf)
12        code.append(neighbor)
13        deg[neighbor] -= 1
14        if deg[neighbor] == 1: leaves.push(neighbor)
15    return code
16
17 function decode_prufer(code, N):
18    deg = count_frequencies(code) + 1 for each node
19    leaves = min_priority_queue([i for i in 1..N if deg[i] == 1])
20    edges = []
21    for c in code:
22        leaf = leaves.pop()
23        edges.append({leaf, c})
24        deg[c] -= 1
25        if deg[c] == 1: leaves.push(c)
26    edges.append({leaves.pop(), leaves.pop()})
27    return edges

```

2.18 Steiner Tree

Complexity: $O(n \cdot 3^k + m \cdot 2^k \log m)$
Minimum Steiner Tree connecting k terminal vertices

```
1 // Complexity: O(N * 3^K + M * 2^K log M) time
2 // Minimum weight connected subgraph containing a designated
  terminal set T of size K <= 12
3
4 function steiner_tree(graph, terminals T):
5     K = len(T)
6     dp[1 << K][N] = INF
7     for i = 0 to K - 1:
8         dp[1 << i][T[i]] = 0
9
10    for mask = 1 to (1 << K) - 1:
11        // Transition 1: Split mask into submasks
12        for sub = (mask - 1) & mask; sub > 0; sub = (sub - 1) &
mask:
13            for u = 1 to N:
14                dp[mask][u] = min(dp[mask][u], dp[sub][u] + dp[mask
^ sub][u])
15
16        // Transition 2: Dijkstra across the graph
17        priority_queue pq
18        for u = 1 to N:
19            if dp[mask][u] < INF: pq.push({dp[mask][u], u})
20        while not pq.empty():
21            d, u = pq.pop()
22            if d > dp[mask][u]: continue
23            for (v, w) in adj[u]:
24                if dp[mask][v] > dp[mask][u] + w:
25                    dp[mask][v] = dp[mask][u] + w
26                    pq.push({dp[mask][v], v})
27
28    return min_{u} dp[(1 << K) - 1][u]
```

2.19 Stable Marriage

Complexity: $O(n^2)$
Gale-Shapley Stable Marriage Problem

```
1 // Complexity: O(N^2) time
2 // Gale-Shapley Algorithm: Finds stable matching between N men and
  N women
3
4 function gale_shapley(men_pref, women_pref):
5     husband_of[1..N] = 0, wife_of[1..N] = 0
6     next_proposal_idx[1..N] = 1
7     free_men_queue = [1..N]
8
9     while not free_men_queue.empty():
10        m = free_men_queue.pop()
11        w = men_pref[m][next_proposal_idx[m]++]
12
13        if husband_of[w] == 0:
14            husband_of[w] = m; wife_of[m] = w
15        else:
16            m_current = husband_of[w]
17            if rank[w][m] < rank[w][m_current]: // w prefers m over
m_current
18                husband_of[w] = m; wife_of[m] = w
19                free_men_queue.push(m_current)
20            else:
21                free_men_queue.push(m)
22
23    return (wife_of, husband_of)
```

3 Tree

3.1 Tree Isomorphism

Complexity: $O(n \log n)$
Tree hashing: Check isomorphic rooted trees or unrooted trees (using centroids as roots)

```
1 // Complexity: O(N log N) or O(N) with randomized hash mapping
2 // Assigns a unique hash to rooted trees. For unrooted trees, run
  on tree centroid(s).
3
4 map<vector<int>, int> subtree_hash_map
5
6 function get_tree_hash(u, parent):
7     child_hashes = []
8     for v in adj[u]:
9         if v == parent: continue
10        child_hashes.append(get_tree_hash(v, u))
11    sort(child_hashes)
12    if child_hashes not in subtree_hash_map:
13        subtree_hash_map[child_hashes] = size(subtree_hash_map) + 1
14    return subtree_hash_map[child_hashes]
15
16 // For unrooted tree comparison:
17 function check_isomorphic(T1, T2):
18     c1 = find_centroids(T1), c2 = find_centroids(T2)
19     if len(c1) != len(c2): return False
20     h1 = get_tree_hash(c1[0], -1)
21     for c in c2:
22         if get_tree_hash(c, -1) == h1: return True
23    return False
```

3.2 Centroid Decomposition

Complexity: $O(n \log n)$
Decompose tree into $O(\log n)$ depth hierarchy for divide-and-conquer path queries

```
1 // Complexity: O(N log N) time, O(N) space
2 // Recursively decompose tree at centroid (size of each component
  <= N / 2)
3
4 bool removed[MAXN]; sz[MAXN]
5
6 function get_sizes(u, p):
7     sz[u] = 1
8     for v in adj[u]:
9         if v != p and not removed[v]:
10            sz[u] += get_sizes(v, u)
11    return sz[u]
12
13 function get_centroid(u, p, total_sz):
14     for v in adj[u]:
15         if v != p and not removed[v] and sz[v] > total_sz / 2:
16            return get_centroid(v, u, total_sz)
17    return u
18
19 function decompose(entry_u, parent_centroid):
20     total_sz = get_sizes(entry_u, -1)
21     c = get_centroid(entry_u, -1, total_sz)
22     removed[c] = True
23     centroid_parent[c] = parent_centroid
24
25     // Process paths passing through centroid c
26     solve_for_centroid(c)
27
28    for v in adj[c]:
```

```
29         if not removed[v]:
30             decompose(v, c)
```

3.3 Heavy-Light Decomposition

Complexity: $O(n \log^2 n)$

Decompose tree into heavy chains for $O(\log^2 n)$ path queries/updates and $O(\log n)$ subtree queries

```
1 // Complexity: O(N) preprocess, O(log^2 N) path query (O(log N)
  with Segment Tree)
2 // Decompose tree into heavy chains where each vertex jumps at most
  O(log N) chains to root
3
4 int parent[N], depth[N], sz[N], heavy[N], head[N], pos[N], cur_pos
  = 0
5
6 function dfs_sz(u, p, d):
7     sz[u] = 1, parent[u] = p, depth[u] = d, heavy[u] = -1,
max_child_sz = 0
8     for v in adj[u]:
9         if v == p: continue
10        dfs_sz(v, u, d + 1)
11        sz[u] += sz[v]
12        if sz[v] > max_child_sz: max_child_sz = sz[v], heavy[u] = v
13
14 function dfs_hld(u, h):
15     head[u] = h, pos[u] = ++cur_pos
16     if heavy[u] != -1: dfs_hld(heavy[u], h)
17     for v in adj[u]:
18         if v != parent[u] and v != heavy[u]: dfs_hld(v, v)
19
20 function query_path(u, v):
21     res = identity
22     while head[u] != head[v]:
23         if depth[head[u]] > depth[head[v]]: swap(u, v)
24         res = combine(res, segtree_query(pos[head[v]], pos[v]))
25         v = parent[head[v]]
26     if depth[u] > depth[v]: swap(u, v)
27     res = combine(res, segtree_query(pos[u], pos[v]))
28    return res
```

4 String

4.1 String Problem Solving Strategies

String problem taxonomy, periodicity, LCP, distinct substrings, palindromes & key theorems

• Problem Taxonomy & Core Approches:

- Longest Common Prefix with Whole String / Single Pattern Match: Use Z-Algorithm or KMP.
- Multiple Dictionary Matching / DP on String Sets: Use Aho-Corasick Automaton.
- Distinct Substrings: Suffix Array ($\frac{n(n+1)}{2} - \sum LCP_i$) or SAM ($\sum len[u] - len[link[u]]$).
- k -th Substring / LCS of Multiple Strings: Use Suffix Automaton (SAM) or Suffix Array + LCP.
- Palindromic Substrings: Use Manacher (radius centered at every index) or EERTREE / PAM (palindromic tree).
- Lexicographically Minimal String Rotation: Run Lyndon Factorization (Duval) on $S + S$.
- Fast Substring Equivalence $O(1)$ / 2D Strings: Use Rolling Hash ($2^{61} - 1$ Mersenne prime).

• Periodicity & Border Properties:

- Minimal period of length- n string is $p = n - \pi[n - 1]$ (if $n \bmod p == 0 \implies S = (S[0..p-1])^{n/p}$).
- Fine-Wilf (Periodicity Lemma):** If S has periods p, q and $|S| \geq p + q - \gcd(p, q) \implies S$ has period $\gcd(p, q)$.
- Border Series:** All borders of a string partition into $\leq O(\log n)$ arithmetic progressions $\implies$ Optimize border DP to $\tilde{O}(n \log n)$.

• Theoretical Bounds:

- Distinct palindromic substrings in length- n string is always $\leq n$.
- Suffix Automaton of length- n string has $\leq 2n - 1$ states and $\leq 3n - 4$ transition edges.

4.2 Rolling Hash (61-bit Mersenne)

Complexity: $O(n)$ build, $O(1)$ substring query

Double/61-bit Mersenne prime Hash ($2^{61} - 1$) with randomized base

```
1 // Complexity: O(N) prep, O(1) substring hash query
2 // Modulo: Mersenne prime 2^61 - 1 (fast bitwise modulo, immune to
  hacking)
3
4 typedef unsigned __int128 uint128;
5 const uint64_t MOD = (1ULL << 61) - 1;
6 uint64_t BASE; // Pick random odd in [256, 10^9] using mt19937_64
7
8 function mul_mod(uint64_t a, uint64_t b):
9     uint128 p = (uint128)a * b;
10    uint64_t res = (p >> 61) + (p & MOD);
11    if (res >= MOD) res -= MOD;
12    return res;
13
14 struct StringHash:
15     vector<uint64_t> H, P; // Prefix hashes and powers of BASE
16
17     function init(string s):
18         n = len(s); H.resize(n + 1, 0); P.resize(n + 1, 1);
19         for i = 0 to n - 1:
20             P[i + 1] = mul_mod(P[i], BASE);
21             H[i + 1] = (mul_mod(H[i], BASE) + s[i]) % MOD;
22
23     // Query hash of substring s[l..r] (0-indexed)
24     function query(l, r):
25         uint64_t sub = mul_mod(H[l], P[r - l + 1]);
26         return (H[r + 1] + MOD - sub) % MOD;
```

4.3 KMP

Complexity: $O(n + m)$

Prefix function π and linear string matching

```
1 // Complexity: O(|P|) preprocess, O(|T|) search
2 // pi[i] = length of longest proper prefix of P[0..i] that is also
  a suffix of P[0..i]
3
4 function compute_pi(pattern P):
5     m = length(P), pi[0] = 0, j = 0
6     for i = 1 to m - 1:
7         while j > 0 and P[i] != P[j]:
8             j = pi[j - 1]
9         if P[i] == P[j]:
10            j = j + 1
11            pi[i] = j
12    return pi
13
14 function kmp_search(text T, pattern P):
```

```
15    pi = compute_pi(P)
16    n = length(T), m = length(P), j = 0
17    for i = 0 to n - 1:
18        while j > 0 and T[i] != P[j]:
19            j = pi[j - 1]
20        if T[i] == P[j]:
21            j = j + 1
22        if j == m:
23            report_match_at(i - m + 1)
24            j = pi[j - 1]
```

4.4 Z-Algorithm

Complexity: $O(n + m)$

Z-array: $z[i] = \text{LCP}(S, S[i..n-1])$ and pattern matching via $S = P + \# + T$

```
1 // Complexity: O(N) time and space
2 // Computes Z-array: z[i] = LCP(S, S[i..N-1])
3 // Recognition: Longest common prefix with entire string, Pattern
  matching via S = P + '#' + T
4
5 function compute_z(string s):
6     n = len(s), z = array of size n (all 0)
7     z[0] = n
8     l = 0, r = 0
9     for i = 1 to n - 1:
10        if i <= r:
11            z[i] = min(r - i + 1, z[i - l])
12        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
13            z[i] += 1
14        if i + z[i] - 1 > r:
15            l = i; r = i + z[i] - 1
16    return z
```

4.5 Aho Corasick

Complexity: $O(\sum |P_i| + |T|)$

Multi-pattern dictionary matching on Trie with BFS failure/exit links

```
1 // Complexity: O(\sum |P_i| * |\Sigma|) build, O(|T|) search
2 // Multi-pattern string matching on Trie with failure & dictionary
  suffix links
3
4 struct Node:
5     next[ALPHABET], link = 0, exit_link = 0, match_ids = []
6
7 function build_aho_corasick(patterns):
8     root = 0, new_node(root)
9     for id, p in enumerate(patterns):
10        u = root
11        for c in p:
12            if not tree[u].next[c]: tree[u].next[c] = new_node()
13            u = tree[u].next[c]
14        tree[u].match_ids.append(id)
15
16    // BFS to build failure links
17    queue Q
18    for c in ALPHABET:
19        v = tree[root].next[c]
20        if v: Q.push(v); tree[v].link = root
21    while not Q.empty():
22        u = Q.pop()
23        for c in ALPHABET:
24            v = tree[u].next[c]
25            if v:
```

```
26        f = tree[u].link
27        while f != root and not tree[f].next[c]: f = tree[f]
28        ].link
29        tree[v].link = tree[f].next[c] if tree[f].next[c]
30        and tree[f].next[c] != v else root
31        tree[v].exit_link = tree[v].link if tree[tree[v].
32        link].match_ids else tree[tree[v].link].exit_link
33        Q.push(v)
34    else:
35        tree[u].next[c] = tree[tree[u].link].next[c] //
36        Trie transition shortcut
```

4.6 Suffix Array

Complexity: $O(n \log n)$ construction, $O(n)$ LCP
Suffix Array doubling construction and Kasai LCP algorithm

```
1 // Complexity: O(N log N) or O(N log^2 N) for SA, O(N) for Kasai
  LCP
2 // SA[i] = start index of i-th lexicographically smallest suffix;
  LCP[i] = lcp(S[SA[i]..], S[SA[i-1]..])
3
4 function build_suffix_array(string S):
5     S = S + "$" // sentinel smallest char
6     n = length(S)
7     SA[0..n-1] = [0..n-1], rank[0..n-1] = ASCII of S
8
9     k = 1
10    while k < n:
11        // Sort SA by key: (rank[i], rank[(i + k) % n])
12        sort(SA, key = (i) -> (rank[i], rank[(i + k) % n]))
13        tmp_rank[SA[0]] = 0
14        for i = 1 to n - 1:
15            prev = SA[i-1], cur = SA[i]
16            same = (rank[cur] == rank[prev] and rank[(cur + k) % n]
17            == rank[(prev + k) % n])
18            tmp_rank[cur] = tmp_rank[prev] + (0 if same else 1)
19            rank = tmp_rank
20            if rank[SA[n-1]] == n - 1: break
21            k = k * 2
22    return SA
23
24 function kasai_lcp(string S, SA):
25     n = length(S), pos[0..n-1], lcp[0..n-1]
26     for i = 0 to n - 1: pos[SA[i]] = i
27     k = 0
28     for i = 0 to n - 1:
29         if pos[i] == 0: continue
30         j = SA[pos[i] - 1]
31         while i + k < n and j + k < n and S[i + k] == S[j + k]: k =
32         k + 1
33         lcp[pos[i]] = k
34         if k > 0: k = k - 1
35    return lcp
```

• LCP Applications & Core CP Recipes:

- Arbitrary Suffix LCP:** $\text{LCP}(S[i..], S[j..]) = \min_{k=\max(pos[i], pos[j])}^{\min(pos[i], pos[j])+1} \text{LCP}[k]$ via RMQ / Sparse Table in $O(1)$ query.
- Distinct Substrings:** Total distinct substrings is $\frac{n(n+1)}{2} - \sum_{i=1}^{n-1} \text{LCP}[i]$.
- k -th Lexicographical Substring:** Suffix $SA[i]$ contributes $n - SA[i] - \text{LCP}[i]$ new strings of lengths $\text{LCP}[i] + 1 \dots n - SA[i]$.
- LCS of 2 Strings:** Build SA on $S_1 + \# + S_2$; ans is $\max \text{LCP}[i]$ where $SA[i], SA[i-1]$ belong to different strings.

- **LCS of K Strings:** Build SA on $S_1 + \#_1 + \dots + S_K + \#_K$; two pointers / sliding window $[L, R]$ covering all K strings $\Rightarrow \min_{k=L+1}^R LCP[k]$.
- **Pattern Occurrences Range:** Suffixes with prefix P form continuous range $[L, R]$ in SA; find via 2 binary searches in $O(|P| \log n)$, count = $R - L + 1$.
- **Longest Substring Occurring $\geq K$ Times:** $\max_i (\min_{j=i+1}^{i+K-1} LCP[j])$ via monotonic queue / sliding RMQ in $O(n)$.

4.7 Suffix Automaton

Complexity: $O(n|\Sigma|)$

Minimal DAWG where every path from root corresponds to a substring of S

```
1 // Complexity: O(N * |\Sigma|) time, O(N * |\Sigma|) space
2 // Directed Acyclic Word Graph (DAWG) representing all substrings
  of S
3
4 struct State:
5     len = 0, link = -1, next[ALPHABET] = {}
6
7 State st[2 * MAXN]; sz = 1, last = 0
8
9 function sam_init():
10     st[0].len = 0, st[0].link = -1; sz = 1, last = 0
11
12 function sam_extend(char c):
13     cur = sz++; st[cur].len = st[last].len + 1
14     p = last
15     while p != -1 and not st[p].next[c]:
16         st[p].next[c] = cur
17         p = st[p].link
18
19     if p == -1:
20         st[cur].link = 0
21     else:
22         q = st[p].next[c]
23         if st[p].len + 1 == st[q].len:
24             st[cur].link = q
25         else:
26             clone = sz++
27             st[clone].len = st[p].len + 1
28             st[clone].next = st[q].next
29             st[clone].link = st[q].link
30             while p != -1 and st[p].next[c] == q:
31                 st[p].next[c] = clone
32                 p = st[p].link
33             st[q].link = st[cur].link = clone
34     last = cur
```

4.8 Lyndon Factorization

Complexity: $O(n)$ time, $O(1)$ space

Duval algorithm: factorize string into Lyndon words $s_1 \geq s_2 \geq \dots \geq s_k$ & Minimal string rotation

```
1 // Complexity: O(N) time, O(1) space
2 // Duval's algorithm: factorizes S into Lyndon words s1 >= s2 >=
  ... >= sk (each si is strictly smaller than all its proper
  suffixes)
3
4 function duval_lyndon_factorization(string S):
5     n = length(S), i = 0, factors = []
6     while i < n:
```

```
7         j = i + 1, k = i
8         while j < n and S[k] <= S[j]:
9             if S[k] < S[j]: k = i
10            else: k = k + 1 // periodic continuation
11            j = j + 1
12        while i <= k:
13            factors.append(S[i .. i + j - k - 1])
14            i = i + j - k
15    return factors
```

4.9 Manacher

Complexity: $O(n)$

Find the longest palindrome radius centered at every index

```
1 // Complexity: O(N) time, O(N) space
2 // Finds longest palindrome radius P[i] centered at i for
  transformed string T = "~#a#b#a#$"
3
4 function manacher(string S):
5     // Insert '#' between characters and '~', '$' as boundary
      markers
6     T = "~#"
7     for c in S: T += "#" + c
8     T += "#$"
9
10    n = length(T), P[0..n-1] = 0
11    C = 0, R = 0 // Center and Right boundary of rightmost
      palindrome
12
13    for i = 1 to n - 2:
14        i_mirror = 2 * C - i
15        if R > i:
16            P[i] = min(R - i, P[i_mirror])
17            // Expand palindrome around i
18            while T[i + 1 + P[i]] == T[i - 1 - P[i]]:
19                P[i] = P[i] + 1
20            // Update center and right boundary
21            if i + P[i] > R:
22                C = i
23                R = i + P[i]
24
25    // Palindrome of max length centered at S[idx]: length is P[i],
      start in S is (i - P[i]) / 2
26    return P
```

4.10 Palindrome Tree

Complexity: $O(n)$

EERTREE: Represent all distinct palindromes in string and transition links

```
1 // Complexity: O(N) time and space
2 // EERTREE stores all distinct palindromic substrings. Root 1: len
  -1, Root 2: len 0.
3
4 struct Node:
5     len, link, next[ALPHABET]
6
7 Node tree[MAXN]; sz, last, S = ""
8
9 function eertree_init():
10    tree[1].len = -1; tree[1].link = 1 // Imaginary root for odd
      length
11    tree[2].len = 0; tree[2].link = 1 // Real root for even length
12    sz = 2, last = 2, S = ""
13
```

```
14 function get_link(p):
15     n = length(S)
16     while n - tree[p].len - 2 < 0 or S[n - tree[p].len - 2] != S[n
      - 1]:
17         p = tree[p].link
18     return p
19
20 function eertree_add_char(char c):
21     S += c
22     cur = get_link(last)
23     if not tree[cur].next[c]:
24         now = ++sz
25         tree[now].len = tree[cur].len + 2
26         tree[now].link = 2 if tree[cur].len == -1 else tree[
      get_link(tree[cur].link)].next[c]
27         tree[cur].next[c] = now
28     last = tree[cur].next[c]
```

4.11 De Bruijn

Complexity: $O(k^n)$

Generate cyclic sequence of length k^n containing all k -ary words of length n

```
1 // Complexity: O(K^N) time and space
2 // Constructs cyclic string of length K^N containing all substrings
  of length N over alphabet [0..K-1]
3
4 function de_bruijn(K, N):
5     if K == 1: return "0" * N
6     visited_edges = set()
7     sequence = []
8
9     function dfs(node_str):
10        for c in 0 to K - 1:
11            edge = node_str + str(c)
12            if edge not in visited_edges:
13                visited_edges.insert(edge)
14                dfs(edge[1..N-1])
15                sequence.append(c)
16
17    start_node = "0" * (N - 1)
18    dfs(start_node)
19    // Repeat prefix of length N - 1 at the end to make it cyclic
20    return reverse(sequence) + start_node
```

5 Dynamic Programming

5.1 Reroot DP

Complexity: $O(n)$ for DFS traversal

Template for Rerooting on Tree (2-pass DFS with Prefix/Suffix transitions)

```
1 // Complexity: O(N) time, O(N) space
2 // Compute subtree values first (DFS1), then reroot to compute all
  roots (DFS2)
3
4 Data dp_down[N], dp_all[N]
5
6 function dfs_down(u, parent):
7     dp_down[u] = base_state(u)
8     for v in adj[u]:
9         if v == parent: continue
10        dfs_down(v, u)
11        dp_down[u] = merge(dp_down[u], transition(dp_down[v], u, v))
```

```

12
13 function dfs_reroot(u, parent, from_parent_val):
14     // Prefix & Suffix merge of children transitions for O(1)
15     // exclusion
16     children = [v for v in adj[u] if v != parent]
17     deg = len(children)
18     pref[0..deg+1] = identity, suff[0..deg+1] = identity
19
20     for i = 1 to deg:
21         pref[i] = merge(pref[i-1], transition(dp_down[children[i-1]], u, children[i-1]))
22     for i = deg down to 1:
23         suff[i] = merge(suff[i+1], transition(dp_down[children[i-1]], u, children[i-1]))
24
25     dp_all[u] = merge(pref[deg], from_parent_val)
26
27     for i = 1 to deg:
28         v = children[i-1]
29         without_v = merge(pref[i-1], suff[i+1])
30         up_to_v = merge(without_v, from_parent_val)
31         dfs_reroot(v, u, transition(up_to_v, v, u))

```

5.2 Divide-And-Conquer DP & Quadrangle Inequality

Complexity: $\mathcal{O}(kn \log n)$ for DnC / $\mathcal{O}(n^2)$ for Knuth Quadrangle Inequality (Monge): $\forall a \leq b \leq c \leq d \implies C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ (Cross $\leq$ Nest).

• **Sufficient & Necessary Local Test:** Checking adjacent 2×2 cells is sufficient (telescoping sum):

$$C(i, j+1) - C(i, j) \geq C(i+1, j+1) - C(i+1, j) \quad (\forall i < j)$$

(Equivalently, cost increase $\Delta_j(i) = C(i, j+1) - C(i, j)$ is non-increasing in i ; or continuous $\frac{\partial^2 C}{\partial x \partial y} \leq 0$).

• **Common Cost Functions Satisfying QI:**

– $C(l, r) = f(S_r - S_{l-1})$ for any convex f ($f'' \geq 0$), e.g. $(S_r - S_{l-1})^2$, $(S_r - S_{l-1})^k$ ($k \geq 1$).

– $C(l, r) = \sum_{k=l}^r w_k = W_r - W_{l-1}$ ($w_k \geq 0$, exact equality = in Knuth Optimal BST).

– $C(l, r) = \text{inv}(l, r)$ (inversion count in subarray $[l, r]$, or any cross-pair property).

– $C(l, r) = |x_r - x_l|^p$ ($p \geq 1$ on sorted x), or variance / sum of squared differences.

• **Monotonicity:** Monge property $\implies \text{opt}(i_1) \leq \text{opt}(i_2)$ for $i_1 \leq i_2$ (DnC: $\mathcal{O}(kn \log n)$) and $\text{opt}[i][j-1] \leq \text{opt}[i][j] \leq \text{opt}[i+1][j]$ (Knuth: $\mathcal{O}(n^2)$).

```

1 // Complexity: O(K * N log N) for DnC, O(N^2) for Knuth
2 // Optimization
3 // Condition: opt[i][j-1] <= opt[i][j] <= opt[i+1][j] (Quadrangle
4 // Inequality)
5 // Recurrence: dp[k][i] = min_{j < i} (dp[k-1][j] + cost(j+1, i))
6
7 function compute(k, L, R, optL, optR):
8     if L > R: return
9     mid = (L + R) / 2
10    best_opt = -1
11    dp[k][mid] = INFINITY
12
13    // Search best transition j in [optL, min(mid-1, optR)]
14    for j = optL to min(mid - 1, optR):

```

```

13    val = dp[k-1][j] + cost(j + 1, mid)
14    if val < dp[k][mid]:
15        dp[k][mid] = val
16        best_opt = j
17
18    // Divide and Conquer to subproblems
19    compute(k, L, mid - 1, optL, best_opt)
20    compute(k, mid + 1, R, best_opt, optR)
21
22    // Knuth's Optimization (O(N^2)):
23    // For len = 2 to N:
24    //     For i = 1 to N - len + 1, j = i + len - 1:
25    //         dp[i][j] = min_{k in [opt[i][j-1], opt[i+1][j]]} (dp[i][k] +
26    //             dp[k+1][j] + cost(i, j))

```

5.3 1D/1D DP Optimization

Complexity: $\mathcal{O}(n \log n)$
Optimize $\text{dp}[i] = \min_{j < i} (\text{dp}[j] + \text{cost}(j, i))$ satisfying quadrangle inequality via Deque of optimal intervals

```

1 // Complexity: O(N log N) time
2 // Recognition: 1D DP dp[i] = min_{j < i} (dp[j] + cost(j, i))
3 // satisfying Quadrangle Inequality
4
5 struct Interval:
6     j, l, r // Candidate j is optimal for all indices in [l, r]
7
8 function optimize_1did(int N):
9     deque<Interval> dq
10    dq.push_back(Interval(0, 1, N))
11    dp[0] = 0
12
13    function eval(j, i): return dp[j] + cost(j, i)
14
15    for i = 1 to N:
16        while not dq.empty() and dq.front().r < i:
17            dq.pop_front()
18        dq.front().l = i
19        dp[i] = eval(dq.front().j, i)
20        if i == N: break
21
22        // Binary search where candidate i beats existing
23        // candidates from back
24        while not dq.empty() and eval(i, dq.back().l) <= eval(dq.back().j, dq.back().l):
25            dq.pop_back()
26
27        if dq.empty():
28            dq.push_back(Interval(i, i + 1, N))
29        else:
30            // Binary search intersection point in dq.back()
31            low = dq.back().l, high = dq.back().r, split = high + 1
32            while low <= high:
33                mid = (low + high) / 2
34                if eval(i, mid) <= eval(dq.back().j, mid):
35                    split = mid; high = mid - 1
36                else: low = mid + 1
37            dq.back().r = split - 1
38            if split <= N:
39                dq.push_back(Interval(i, split, N))
40
41    return dp[N]

```

5.4 Alien DP

Complexity: $\mathcal{O}(n \log(\max_val))$

Dynamic Programming on convex/concave functions (WQS Binary Search):

- Cost is convex/concave $\rightarrow$ Greedy selection.
- DP is convex/concave $\rightarrow$ Binary search penalty λ .

```

1 // Complexity: O(N log(C))
2 // Problem: Optimize f(K) = min cost choosing exactly K items,
3 // where f is convex/concave.
4 // Trick: Introduce penalty lambda for each item chosen: g(lambda)
5 // = min_x (cost(x) - lambda * cnt(x))
6
7 function solve_with_penalty(lambda):
8     // dp[i] stores pair (optimal_cost, count_of_items_chosen)
9     dp[0] = (0, 0)
10    for i = 1 to N:
11        dp[i] = dp[i-1] // case: don't choose item i
12        take_cost = dp[i-1].cost + value[i] - lambda
13        take_cnt = dp[i-1].cnt + 1
14        if take_cost < dp[i].cost: // or <= depending on tie-breaking
15            dp[i] = (take_cost, take_cnt)
16    return dp[N]
17
18 function aliens_trick(target_K):
19    low = -MAX_VAL, high = MAX_VAL, best_lambda = 0
20    while low <= high:
21        mid = (low + high) / 2
22        res = solve_with_penalty(mid)
23        if res.cnt >= target_K:
24            best_lambda = mid
25            high = mid - 1 // or low = mid + 1 for max problem
26        else:
27            low = mid + 1
28    ans = solve_with_penalty(best_lambda)
29    return ans.cost + target_K * best_lambda

```

5.5 Convex Hull Trick

Complexity: $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$ with monotonic slopes
Dynamic Programming optimization for linear functions $y = mx + c$

```

1 // Complexity: O(N log N) or O(N) if slopes are monotonic
2 // Form: dp[i] = min_{j < i} (m[j] * x[i] + c[j])
3
4 struct Line:
5     m, c // y = m * x + c
6
7 // Intersection x-coordinate between line 1 and line 2
8 function intersect_x(l1, l2):
9     return (l2.c - l1.c) / (l1.m - l2.m)
10
11 // Monotonic Slope Deque / Stack (O(N)):
12 deque<Line> hull
13
14 function add_line(new_line):
15     // If slopes decreasing and finding min:
16     while size(hull) >= 2 and intersect_x(hull[last], new_line) <=
17         intersect_x(hull[last-1], hull[last]):
18         hull.pop_back()
19     hull.push_back(new_line)
20
21 function query_min(x):
22     // Two pointers (if query x is increasing):
23     while size(hull) >= 2 and hull[0].eval(x) >= hull[1].eval(x):
24         hull.pop_front()
25     return hull[0].eval(x)
26
27 // Or binary search on hull if x is arbitrary (O(log N))

```

5.6 Slope Trick

Complexity: $\mathcal{O}(n \log n)$

Maintain continuous piecewise linear convex function $f(x)$ with $+|x - a|$, prefix min, domain shift via 2 Heaps

```
1 // Complexity:  $\mathcal{O}(N \log N)$  time
2 // Recognition: DP with convex piecewise linear function,  $f(x) += |x - a|$ , prefix min
3
4 struct SlopeTrick:
5     max_pq L; min_pq R // Multiset of inflection points
6     min_f = 0, l_add = 0, r_add = 0
7
8     // Add  $|x - a|$  to function  $f(x)$ 
9     function add_abs(a):
10         l_max = L.empty() ? -INF : L.top() + l_add
11         r_min = R.empty() ? INF : R.top() + r_add
12         if a < l_max:
13             min_f += l_max - a
14             L.pop(); L.push(a - l_add); L.push(a - l_add)
15             R.push(l_max - r_add)
16         else if a > r_min:
17             min_f += a - r_min
18             R.pop(); R.push(a - r_add); R.push(a - r_add)
19             L.push(r_min - l_add)
20         else:
21             L.push(a - l_add); R.push(a - r_add)
22
23     // Prefix minimum:  $g(x) = \min_y \{x \leq y\} f(y)$  (clears right slopes)
24     function prefix_min():
25         R.clear()
26
27     // Shift function domain  $[x - b, x + a]$ :  $g(x) = \min_{\{x - b \leq y \leq x + a\}} f(y)$ 
28     function shift(a, b):
29         l_add -= b; r_add += a
```

5.7 SOS DP

Complexity: $\mathcal{O}(N \cdot 2^N)$

Sum Over Subsets / Fast Zeta Transform: Compute sum over submasks / supermasks in $\mathcal{O}(N \cdot 2^N)$ ($N \leq 24$)

```
1 // Complexity:  $\mathcal{O}(N * 2^N)$  time,  $\mathcal{O}(2^N)$  space ( $N \leq 24$ )
2 // Recognition: Sum over subsets/supersets, counting pairs with  $[A[i] \text{ AND } A[j]] == 0$ 
3
4 function sos_dp(int N, array F):
5     // F[mask] initialized with base values
6     // Sum over all submasks:  $F[\text{mask}] = \sum_{\text{sub} \subseteq \text{mask}} A[\text{sub}]$ 
7     for i = 0 to N - 1:
8         for mask = 0 to  $(1 \ll N) - 1$ :
9             if mask & (1 << i):
10                 F[mask] += F[mask ^ (1 << i)]
11
12     // Sum over all supermasks:  $G[\text{mask}] = \sum_{\text{mask} \subseteq \text{sup}} A[\text{sup}]$ 
13     // for i = 0 to N - 1:
14     //     for mask =  $(1 \ll N) - 1$  down to 0:
15     //         if not (mask & (1 << i)):
16     //             G[mask] += G[mask | (1 << i)]
17     return F
```

5.8 Monotonic Queue Optimization

Complexity: $\mathcal{O}(1)$ amortized per push/pop

Deque / Two-Stack Sliding Window minimum/maximum

```
1 // Complexity:  $\mathcal{O}(1)$  amortized per push/pop/get_min
2 // Two-stack Min Queue or Monotonic Deque for Sliding Window
3
4 struct MinQueue:
5     deque<pair<value, index>> dq
6
7     function push(val, idx):
8         while not dq.empty() and dq.back().value >= val:
9             dq.pop_back()
10        dq.push_back({val, idx})
11
12    function pop(min_valid_idx):
13        while not dq.empty() and dq.front().index < min_valid_idx:
14            dq.pop_front()
15
16    function get_min():
17        return dq.front().value
```

5.9 Expected Value Trick

Complexity: $\mathcal{O}(n\sqrt{n})$

Suppose we optimize picking k objects from n randomly shuffled items: at step i , only consider state range $\left[\frac{i-k}{n} - C\sqrt{n}, \frac{i-k}{n} + C\sqrt{n}\right]$.

```
1 // Complexity:  $\mathcal{O}(N * \text{sqrt}(N))$ 
2 // Problem: Pick K items from N shuffled items optimizing cost.
3 // Key Insight: The number of items chosen after prefix i concentrates around  $E[X] = (i * K) / N$ .
4 // With high probability, optimal count at step i lies in  $[(i*K)/N - C*\text{sqrt}(N), (i*K)/N + C*\text{sqrt}(N)]$ .
5
6 function solve_ev_dp(N, K):
7     dp[0..1][0..K] = -INFINITY
8     dp[0][0] = 0
9
10    for i = 1 to N:
11        cur = i % 2, prev = 1 - cur
12        fill(dp[cur], -INFINITY)
13
14        expected_cnt = (i * K) / N
15        low_cnt = max(0, expected_cnt - C * sqrt(N))
16        high_cnt = min(min(i, K), expected_cnt + C * sqrt(N))
17
18        for j = low_cnt to high_cnt:
19            // Transition 1: don't take item i
20            dp[cur][j] = max(dp[cur][j], dp[prev][j])
21            // Transition 2: take item i
22            if j > 0 and dp[prev][j-1] != -INFINITY:
23                dp[cur][j] = max(dp[cur][j], dp[prev][j-1] + val[i])
24
25    return dp[N % 2][K]
```

6 Math

6.1 Number Theory Formulas

Modular arithmetic, Extended Euler Power Reduction, Legendre, Wilson, Frobenius Coin, Floor Trick, CRT, Fibonacci

- Extended Euler Totient Power Reduction ($\forall a, m$):

$$a^b \equiv a^{(b \bmod \phi(m)) + \phi(m)} \pmod{m} \quad (b \geq \phi(m))$$

- Legendre's Formula ($v_p(n!)$): $v_p(n!) = \sum_{k=1}^{\infty} \lfloor \frac{n}{p^k} \rfloor = \frac{n - S_p(n)}{p-1}$.
- Wilson's Theorem: $(p-1)! \equiv -1 \pmod{p} \iff p$ prime $\implies (p-2)! \equiv 1 \pmod{p}$.
- Frobenius Coin (Chicken McNugget): For $\gcd(a, b) = 1$, max non-representable $ax+by$ ($x, y \geq 0$) is $ab-a-b$; total non-representable count is $\frac{(a-1)(b-1)}{2}$.
- Floor Trick ($\mathcal{O}(\sqrt{n})$ Blocks):

```
1 for (int l = 1, r; l <= n; l = r + 1) {
2     int v = n / l; r = n / v;
3 }
```

- Chinese Remainder Theorem (CRT): $x \equiv a_i \pmod{m_i}$.

$$x \equiv \sum_{i=1}^k a_i M_i (M_i^{-1} \bmod m_i) \pmod{M} \quad (M = \prod m_i, M_i = M/m_i)$$

- Fibonacci: $\gcd(F_m, F_n) = F_{\gcd(m, n)}$, $F_{2n} = F_n(2F_{n+1} - F_n)$.

6.2 Like-Euclid Floor Sum

Complexity: $\mathcal{O}(\log m)$

Compute $\sum_{i=0}^{n-1} \lfloor \frac{ai+b}{m} \rfloor$ via coordinate transformation

```
1 // Complexity:  $\mathcal{O}(\log m)$ 
2 // Computes  $\sum_{i=0}^{n-1} \lfloor \frac{a * i + b}{m} \rfloor$  (Like-Euclid)
3 // Recognition: Summing floor divisions of linear functions over ranges
4
5 function floor_sum(n, m, a, b):
6     ans = 0
7     if a >= m:
8         ans += (n - 1) * n / 2 * (a / m)
9         a %= m
10    if b >= m:
11        ans += n * (b / m)
12        b %= m
13    y_max = (a * n + b) / m
14    x_max = y_max * m - b
15    if y_max == 0: return ans
16    // Rotate coordinate system: sum over y from 0 to y_max - 1
17    ans += (n - (x_max + a - 1) / a) * y_max
18    ans += floor_sum(y_max, a, m, (a - x_max % a) % a)
19    return ans
```

6.3 Rabin-Miller

Complexity: $\mathcal{O}(k \log n)$

Deterministic Miller-Rabin prime checking for $n < 2^{64}$

```
1 // Complexity:  $\mathcal{O}(k \log^3 n)$  or  $\mathcal{O}(k \log n)$  with fast multiplication
2 // Deterministic for  $n < 2^{64}$  using bases: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}
3
4 function power_mod(a, b, m):
5     res = 1; a %= m
6     while b > 0:
7         if b & 1: res = (res * a) % m
```

```

8     a = (a * a) % m; b >= 1
9     return res
10
11 function miller_test(d, n, a):
12     x = power_mod(a, d, n)
13     if x == 1 or x == n - 1: return True
14     while d != n - 1:
15         x = (x * x) % n; d *= 2
16         if x == n - 1: return True
17         if x == 1: return False
18     return False
19
20 function is_prime(n):
21     if n < 2: return False
22     if n <= 3: return True
23     if n % 2 == 0: return False
24     d = n - 1
25     while d % 2 == 0: d /= 2
26     for a in [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]:
27         if n == a: return True
28         if not miller_test(d, n, a): return False
29     return True

```

6.4 Fast Sieve

Complexity: $O(N)$

Linear Euler Sieve computing primes and smallest prime factors up to N

```

1 // Complexity:  $O(N)$  time and  $O(N)$  space (Linear Euler Sieve)
2 // Finds all primes up to  $N$  and smallest prime factor min_prime[i]
3
4 int min_prime[N], primes[]
5
6 function linear_sieve(N):
7     for i = 2 to N:
8         if min_prime[i] == 0:
9             min_prime[i] = i
10            primes.append(i)
11        for p in primes:
12            if p > min_prime[i] or i * p > N: break
13            min_prime[i * p] = p
14    return primes

```

6.5 Prime Counting

Complexity: $O(N^{2/3})$ or $O(N^{3/4})$

Lucy Hedgehog / Meissel-Lehmer: Count primes $\leq N$

```

1 // Complexity:  $O(N^{3/4})$  or  $O(N^{2/3})$  with Lucy Hedgehog
  algorithm
2 // Counts number of primes  $\leq N$  without generating all primes
3
4 function count_primes(N):
5     sq = sqrt(N)
6     V = [N / i for i in 1..N] // All distinct values of floor(N / i)
7     // S[v] stores number of integers  $\leq v$  with prime factors  $\geq p$ 
8     S = map()
9     for v in V: S[v] = v - 1
10
11    for p = 2 to sq:
12        if S[p] > S[p - 1]: // p is prime
13            sp = S[p - 1]
14            p2 = p * p
15            for v in V:
16                if v < p2: break

```

```

17    S[v] -= S[v / p] - sp
18
19    return S[N]

```

6.6 Pollard Rho

Complexity: $O(\sqrt[4]{n} \log n)$

Find non-trivial factors of composite numbers with Brent's cycle detection

```

1 // Complexity:  $O(N^{1/4} \log N)$  expected time
2 // Finds a non-trivial factor of composite number  $N$  using Brent's
  cycle detection
3
4 function pollard_rho(N):
5     if N % 2 == 0: return 2
6     x = rand() % (N - 2) + 2, y = x
7     c = rand() % (N - 1) + 1, d = 1
8
9     f = (v) -> (mul_mod(v, v, N) + c) % N
10
11    while d == 1:
12        x = f(x)
13        y = f(f(y))
14        d = gcd(abs(x - y), N)
15    if d == N: return pollard_rho(N) // Retry with different
      random c
16    return d
17
18 function factorize(N, factors_list):
19     if N == 1: return
20     if is_prime(N): factors_list.append(N); return
21     d = pollard_rho(N)
22     factorize(d, factors_list)
23     factorize(N / d, factors_list)

```

6.7 Primitive Root

Complexity: $O(\text{ans} \cdot \log p)$

Find smallest primitive root g modulo prime p

```

1 // Complexity:  $O(\text{Ans} \cdot \log(P) \cdot \text{num\_factors})$  time
2 // Finds smallest primitive root modulo  $P$  (prime)
3
4 function find_primitive_root(P):
5     phi = P - 1
6     factors = unique_prime_factors(phi)
7
8     for g = 2 to P - 1:
9         is_primitive = True
10        for p in factors:
11            if power_mod(g, phi / p, P) == 1:
12                is_primitive = False
13                break
14        if is_primitive: return g
15    return -1

```

6.8 Baby-step Giant-Step

Complexity: $O(\sqrt{m} \log m)$

Find smallest x such that $a^x \equiv b \pmod{m}$ (Discrete Log)

```

1 // Complexity:  $O(\sqrt{M} \log M)$  time and  $O(\sqrt{M})$  space
2 // Finds smallest non-negative integer  $x$  such that  $a^x \equiv b \pmod{M}$ 
3
4 function baby_step_giant_step(a, b, M):

```

```

5     a %= M; b %= M
6     k = 1, add = 0, g = gcd(a, M)
7     while g > 1:
8         if b == k: return add
9         if b % g != 0: return -1 // No solution
10        b /= g; M /= g; add += 1
11        k = (k * (a / g)) % M
12        g = gcd(a, M)
13
14    m = ceil(sqrt(M))
15    table = hash_map() // Stores  $a^k(j * m) * k \pmod{M} \rightarrow j$ 
16    cur = 1
17    for i = 0 to m - 1:
18        if cur not in table: table[(cur * b) % M] = i
19        cur = (cur * a) % M
20
21    base = cur //  $a^m \pmod{M}$ 
22    cur = k
23    for j = 1 to m:
24        cur = (cur * base) % M
25        if cur in table:
26            return j * m - table[cur] + add
27    return -1

```

6.9 Sqrt Mod

Complexity: $O(\log^2 p)$

Tonelli-Shanks: Solve $x^2 \equiv a \pmod{p}$

```

1 // Complexity:  $O(\log^2 P)$  expected time
2 // Tonelli-Shanks Algorithm: Solves  $x^2 \equiv a \pmod{P}$  for odd prime  $P$ 
3
4 function tonelli_shanks(a, P):
5     if a == 0: return 0
6     if power_mod(a, (P - 1) / 2, P) != 1: return -1 // No quadratic
      residue
7     if P % 4 == 3: return power_mod(a, (P + 1) / 4, P)
8
9     //  $P - 1 = Q \cdot 2^S$  with  $Q$  odd
10    S = 0, Q = P - 1
11    while Q % 2 == 0: S += 1; Q /= 2
12
13    // Find quadratic non-residue z
14    z = 2
15    while power_mod(z, (P - 1) / 2, P) == 1: z += 1
16
17    M = S, c = power_mod(z, Q, P), t = power_mod(a, Q, P), R =
      power_mod(a, (Q + 1) / 2, P)
18    while t != 1:
19        i = 1, cur = (t * t) % P
20        while cur != 1: cur = (cur * cur) % P; i += 1
21        b = power_mod(c, 1 << (M - i - 1), P)
22        M = i; c = (b * b) % P; t = (t * c) % P; R = (R * b) % P
23    return R

```

6.10 Extended GCD

Complexity: $O(\log \min(a, b))$

Find x, y that satisfy the equation $ax + by = \gcd(a, b)$

```

1 // Complexity:  $O(\log(\min(a, b)))$ 
2 // Finds  $x, y$  such that  $a*x + b*y = \gcd(a, b)$ 
3
4 function extgcd(a, b):
5     if b == 0: return (g = a, x = 1, y = 0)
6     res = extgcd(b, a % b)
7     return (g = res.g, x = res.y, y = res.x - (a / b) * res.y)

```

```

8
9 function mod_inverse(a, m):
10     res = extgcd(a, m)
11     if res.g != 1: return -1 // Inverse does not exist
12     return (res.x % m + m) % m

```

6.11 Diophantine

Complexity: $O(\log \min(a, b))$

General solutions to linear Diophantine equation $ax + by = c$

```

1 // Complexity: O(log(min(a, b)))
2 // Solves a*x + b*y = c. General solutions: x = x0 + k*(b/g), y =
  y0 - k*(a/g)
3
4 function solve_diophantine(a, b, c):
5     res = extgcd(abs(a), abs(b))
6     g = res.g
7     if c % g != 0: return (has_sol = False, 0, 0)
8     x0 = res.x * (c / g)
9     y0 = res.y * (c / g)
10    if a < 0: x0 = -x0
11    if b < 0: y0 = -y0
12    return (has_sol = True, x0, y0, g)

```

6.12 Stern-Brocot Tree

Complexity: $\mathcal{O}(\log(p + q))$

Fractions in reduced forms, tree encoding/decoding, LCA

```

1 // Complexity: O(log(p + q))
2 // Binary search on Stern-Brocot tree to encode/decode positive
  reduced fractions p / q
3
4 function encode_path(p, q):
5     path = []
6     L_num = 0, L_den = 1; R_num = 1, R_den = 0
7     while True:
8         M_num = L_num + R_num, M_den = L_den + R_den
9         if p * M_den == q * M_num: break
10        if p * M_den < q * M_num: // Go Left
11            k = (q * L_num - p * L_den) / (p * R_den - q * R_num)
12        // accelerate steps
13        k = max(1, k)
14        path.append({'L', k})
15        R_num += k * L_num; R_den += k * L_den
16    else: // Go Right
17        k = (p * R_den - q * R_num) / (q * L_num - p * L_den)
18        k = max(1, k)
19        path.append({'R', k})
20        L_num += k * R_num; L_den += k * R_den
21    return path

```

6.13 Multiplicative Functions & Möbius Inversion

Dirichlet Convolution, Möbius Inversion, GCD/LCM summations

• **Euler Phi & Möbius:** $\phi(n) = n \prod_{p|n} (1 - 1/p)$, $\sum_{d|n} \phi(d) = n$, $\sum_{d|n} \mu(d) = [n = 1]$.

• **Dirichlet & Inversion:** $(f * g)(n) = \sum_{d|n} f(d)g(n/d)$.

$$f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu(d)f(n/d)$$

• **Count coprime pairs:**

$$\sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = 1] = \sum_{d=1}^{\min(n, m)} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

• **Sum of GCD over all pairs:**

$$\sum_{i=1}^n \sum_{j=1}^m \gcd(i, j) = \sum_{g=1}^{\min(n, m)} \phi(g) \left\lfloor \frac{n}{g} \right\rfloor \left\lfloor \frac{m}{g} \right\rfloor$$

• **Sum of LCM over all pairs** ($S(x) = \frac{x(x+1)}{2}$):

$$\sum_{i \leq n, j \leq m} \text{lcm}(i, j) = \sum_g g \sum_d d^2 \mu(d) S\left(\left\lfloor \frac{n}{gd} \right\rfloor\right) S\left(\left\lfloor \frac{m}{gd} \right\rfloor\right)$$

6.14 Combinatorics Formulas

Binomial identities, Hockey-stick, Vandermonde, Inversion, Stars and bars, Catalan, Stirling, Cayley, LGV

• **Binomial Identities:** $\binom{n}{k} = \binom{n}{n-k} = \binom{n-1}{k-1} + \binom{n-1}{k}$; $k \binom{n}{k} = n \binom{n-1}{k-1}$; $\frac{1}{k+1} \binom{n}{k} = \frac{1}{n+1} \binom{n+1}{k+1}$; $\binom{n}{k} \binom{k}{m} = \binom{n}{m} \binom{n-m}{k-m}$; Upper negation: $\binom{-n}{k} = (-1)^k \binom{n+k-1}{k}$.

• **Binomial Summations:** $\sum_{k=0}^n \binom{n}{k} = 2^n$; $\sum_{k=0}^n (-1)^k \binom{n}{k} = [n = 0]$; $\sum_k \text{even} \binom{n}{k} = \sum_k \text{odd} \binom{n}{k} = 2^{n-1}$; $\sum_{k=0}^n k \binom{n}{k} = n 2^{n-1}$; $\sum_{k=0}^n k^2 \binom{n}{k} = n(n+1) 2^{n-2}$; $\sum_{k=0}^n \frac{1}{k+1} \binom{n}{k} = \frac{2^{n+1}-1}{n+1}$.

• **Hockey-Stick & Vandermonde:** $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$; $\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k+1}$; $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r} \implies \sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$.

• **Binomial Inversion & Differences:** $f(n) = \sum_{k=0}^n \binom{n}{k} g(k) \iff g(n) = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} f(k)$; Upper: $f(n) = \sum_{k=n}^N \binom{k}{n} g(k) \iff g(n) = \sum_{k=n}^N (-1)^{k-n} \binom{k}{n} f(k)$; $\sum_{k=0}^n (-1)^{n-k} \binom{n}{k} k^m = n!$ if $m < n$, $n!$ if $m = n$.

• **Stars and Bars:** $x_i \geq 0 \implies \binom{n+k-1}{k-1}$; $x_i \geq 1 \implies \binom{n-1}{k-1}$; $x_i \geq c_i \implies n' = n - \sum c_i$.

• **Derangements:** $D_0 = 1, D_1 = 0, D_n = (n-1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n \approx [n!/e]$.

• **Catalan Numbers** $C_n = \frac{1}{n+1} \binom{2n}{n}$: Valid bracketings, Full binary trees with $n+1$ leaves, Triangulations of convex $(n+2)$ -gon, Dyck paths not crossing $y = x$, 1-Stack permutations.

• **Reflection Principle:** Paths $(0, 0) \rightarrow (n, m)$ not touching $y = x + k$ ($k > 0$): $\binom{n+m}{m} - \binom{n+m}{m-k}$.

• **Balloting Theorem:** A has a votes, B has b ($a > b$). Prob A strictly leads throughout: $\frac{a-b}{a+b}$.

• **Stirling Numbers of 1st & 2nd Kind:** $\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix}$, $\begin{Bmatrix} n \\ k \end{Bmatrix} = \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} + k \begin{Bmatrix} n-1 \\ k \end{Bmatrix}$. $x^n = \sum_{k=0}^n \begin{Bmatrix} n \\ k \end{Bmatrix} x^{\underline{k}} \implies \sum_{i=1}^n i^k = \sum_{j=0}^k \begin{Bmatrix} k \\ j \end{Bmatrix} j! \binom{n+1}{j+1}$.

• **PIE:** $|\bigcup_{i=1}^n A_i| = \sum_{\emptyset \neq J} (-1)^{|J|-1} |\bigcap_{j \in J} A_j|$; Exactly r sets: $\sum_{m=r}^n (-1)^{m-r} \binom{m}{r} \sum_{|X|=m} |\bigcap_{i \in X} A_i|$.

• **Lucas' Theorem:** $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$ (base- p digits).

• **Tree Counting & Cayley:** n^{n-2} labeled trees, Forest of k rooted trees kn^{n-k-1} , Matrix Tree Theorem ($\det(L_i)$), LGV Lemma ($\det(M)$).

6.15 Generating Functions & Taylor Series

OGF, EGF, Taylor expansions, Simpson numerical integration

• OGF: $A(x) = \sum a_n x^n$; EGF: $A(x) = \sum a_n \frac{x^n}{n!}$.

• $\frac{1}{1-x} = \sum_{n \geq 0} x^n$, $\frac{1}{(1-x)^k} = \sum_{n \geq 0} \binom{n+k-1}{k-1} x^n$.

• $e^x = \sum_{n \geq 0} \frac{x^n}{n!}$, $\ln(1-x) = -\sum_{n \geq 1} \frac{x^n}{n}$, $(1+x)^\alpha = \sum_{n \geq 0} \binom{\alpha}{n} x^n$.

• **Simpson's Rule:** $\int_a^{a+2h} f(x) dx \approx \frac{h}{3} (f(a) + 4f(a+h) + f(a+2h))$.

6.16 Linear Recurrences (Berlekamp-Massey)

Complexity: $\mathcal{O}(N^2)$ find recurrence, $\mathcal{O}(N^2 \log K)$ compute k -th term

Find minimal linear recurrence from first $2N$ terms and compute K -th term via $x^K \bmod P(x)$

```

1 // Complexity: O(N^2) to find recurrence, O(N^2 log K) or O(N log N
  log K) for K-th term
2 // Finds minimal linear recurrence C for sequence S: S[i] = \sum_{j
  =1}^m C[j] * S[i - j]
3
4 function berlekamp_massey(sequence S):
5     C = [1], B = [1] // current and backup polynomial
6     L = 0, m = 1, b = 1
7
8     for i = 0 to len(S) - 1:
9         d = S[i] // discrepancy
10        for j = 1 to L: d = (d + C[j] * S[i - j]) % MOD
11        if d == 0:
12            m += 1
13        else:
14            T = C
15            coef = (d * inv(b)) % MOD
16            pad_B = [0] * m + [(-coef * x) % MOD for x in B]
17            C = poly_add(C, pad_B)
18            if 2 * L <= i:
19                L = i + 1 - L; B = T; b = d; m = 1
20            else:
21                m += 1
22        return [-c for c in C[1..L]] // recurrence coefficients
23
24 function linear_recurrence_kth(C, S, K):
25     // Polynomial modulus: compute x^K mod (x^L - \sum C[j] x^{L-j}
  ) with binary exponentiation
26     P = poly_pow_mod([0, 1], K, char_poly(C))
27     return \sum_{i=0}^{L-1} P[i] * S[i]

```

6.17 FWHT

Complexity: $\mathcal{O}(n \log n)$

Fast Walsh-Hadamard Transform (XOR, AND, OR Convolution)

```
1 // Complexity:  $\mathcal{O}(N \log N)$  where  $N = 2^k$ 
2 // Fast Walsh-Hadamard Transform for bitwise XOR convolution ( $C[k] = \sum_{i \sim j, k = k} A[i] * B[j]$ )
3
4 function fwht_xor(a, invert):
5     n = len(a)
6     len_step = 1
7     while len_step < n:
8         for i = 0 to n - 1 step 2 * len_step:
9             for j = 0 to len_step - 1:
10                 u = a[i + j], v = a[i + len_step + j]
11                 a[i + j] = (u + v) % MOD
12                 a[i + len_step + j] = (u - v + MOD) % MOD
13             len_step *= 2
14     if invert:
15         inv_n = power_mod(n, MOD - 2, MOD)
16         for i = 0 to n - 1: a[i] = (a[i] * inv_n) % MOD
```

6.18 Lagrange

Complexity: $\mathcal{O}(n)$ evaluation

Lagrange polynomial interpolation at point X on $n + 1$ equidistant points $y_0, \dots, y_n$

```
1 // Complexity:  $\mathcal{O}(N)$  evaluation at point  $X$  for consecutive points
   y_0 = f(0), y_1 = f(1), ..., y_n = f(n)
2 // Formula:  $f(X) = \sum_{i=0}^n y_i \prod_{j \neq i} \frac{X - j}{i - j}$ 
3
4 function lagrange_eval_consecutive(y[0..n], X):
5     if X <= n: return y[X]
6     pref[0..n], suff[0..n]
7     pref[0] = X, suff[n] = (X - n) % MOD
8     for i = 1 to n: pref[i] = (pref[i-1] * (X - i)) % MOD
9     for i = n - 1 down to 0: suff[i] = (suff[i+1] * (X - i)) % MOD
10
11     ans = 0
12     for i = 0 to n:
13         num = (1 if i == 0 else pref[i-1]) * (1 if i == n else suff[i+1]) % MOD
14         den = (fact[i] * fact[n - i]) % MOD
15         term = (y[i] * num % MOD) * inv(den) % MOD
16         if (n - i) % 2 == 1: ans = (ans - term + MOD) % MOD
17         else: ans = (ans + term) % MOD
18     return ans
```

6.19 FFT, NTT, and Polynomials

Complexity: $\mathcal{O}(n \log n)$ for each operation

Compact NTT / formal power series template for 998244353: convolution, inverse, log, exp, power, division and modulo

```
1 // Polynomial / FPS for MOD = 998244353 (primitive root = 3).
2 // Coefficient i of Poly p is p[i]. All operations are truncated
   to n when needed.
3 const int MOD = 998244353;
4
5 struct mint {
6     int v;
7     mint(long long x = 0) : v((x % MOD + MOD) % MOD) {}
8     mint pow(long long e) const {
9         mint a = *this, r = 1;
10         for (; e >= 1, a *= a; if (e & 1) r *= a;
```

```
11         return r;
12     }
13     mint inv() const { return pow(MOD - 2); }
14     mint& operator += (mint o) { return v = (v += o.v) >= MOD ? v - MOD : v, *this; }
15     mint& operator -= (mint o) { return v = (v -= o.v) < 0 ? v + MOD : v, *this; }
16     mint& operator *= (mint o) { return v = 1LL * v * o.v % MOD, *this; }
17     mint& operator /= (mint o) { return *this *= o.inv(); }
18     friend mint operator + (mint a, mint b) { return a + b; }
19     friend mint operator - (mint a, mint b) { return a - b; }
20     friend mint operator * (mint a, mint b) { return a * b; }
21     friend mint operator / (mint a, mint b) { return a / b; }
22     friend bool operator == (mint a, mint b) { return a.v == b.v; }
23     friend bool operator != (mint a, mint b) { return a.v != b.v; }
24 };
25
26 using Poly = vector<mint>;
27
28 void ntt(Poly& a, bool invert) {
29     int n = (int)a.size();
30     for (int i = 1, j = 0; i < n; ++i) {
31         int bit = n >> 1;
32         for (; j & bit; bit >>= 1) j ^= bit;
33         j ^= bit;
34         if (i < j) swap(a[i], a[j]);
35     }
36     for (int len = 2; len <= n; len <= 1) {
37         mint wlen = mint(3).pow((MOD - 1) / len);
38         if (invert) wlen = wlen.inv();
39         for (int i = 0; i < n; i += len) {
40             mint w = 1;
41             for (int j = 0; j < len / 2; ++j, w *= wlen) {
42                 mint u = a[i + j], v = a[i + j + len / 2] * w;
43                 a[i + j] = u + v;
44                 a[i + j + len / 2] = u - v;
45             }
46         }
47     }
48     if (invert) {
49         mint inv_n = mint(n).inv();
50         for (mint& x : a) x *= inv_n;
51     }
52 }
53
54 Poly operator * (Poly a, Poly b) {
55     if (a.empty() || b.empty()) return {};
56     int need = (int)a.size() + (int)b.size() - 1, n = 1;
57     while (n < need) n <= 1;
58     a.resize(n); b.resize(n);
59     ntt(a, false); ntt(b, false);
60     for (int i = 0; i < n; ++i) a[i] *= b[i];
61     ntt(a, true); a.resize(need);
62     return a;
63 }
64
65 void norm(Poly& a) { while (!a.empty() && a.back() == 0) a.pop_back(); }
66 Poly cut(Poly a, int n) { a.resize(min((int)a.size(), n)); return a; }
67
68 Poly operator + (Poly a, const Poly& b) {
69     a.resize(max(a.size(), b.size()));
70     for (int i = 0; i < (int)b.size(); ++i) a[i] += b[i];
71     return a;
72 }
73 Poly operator - (Poly a, const Poly& b) {
74     a.resize(max(a.size(), b.size()));
```

```
75     for (int i = 0; i < (int)b.size(); ++i) a[i] -= b[i];
76     return a;
77 }
78 Poly operator * (Poly a, mint k) { for (mint& x : a) x *= k; return a; }
79 Poly operator * (mint k, Poly a) { return a * k; }
80
81 Poly derivative(const Poly& a) {
82     Poly r(max(0, (int)a.size() - 1));
83     for (int i = 1; i < (int)a.size(); ++i) r[i - 1] = a[i] * i;
84     return r;
85 }
86 Poly integral(const Poly& a) {
87     Poly r(a.size() + 1);
88     for (int i = 0; i < (int)a.size(); ++i) r[i + 1] = a[i] / (i + 1);
89     return r;
90 }
91
92 Poly inverse(const Poly& a, int n) { // requires a[0] != 0
93     Poly r{a[0].inv()};
94     for (int m = 1; m < n; m <= 1) {
95         Poly f = cut(a, 2 * m);
96         r = cut(r * (Poly{2} - cut(f * r, 2 * m)), 2 * m);
97     }
98     return cut(r, n);
99 }
100
101 Poly logarithm(const Poly& a, int n) { // requires a[0] == 1
102     return cut(integral(cut(derivative(a) * inverse(a, n), n - 1)), n);
103 }
104
105 Poly exponential(const Poly& a, int n) { // requires a[0] == 0
106     Poly r{1};
107     for (int m = 1; m < n; m <= 1) {
108         Poly q = cut(a, 2 * m) - logarithm(r, 2 * m);
109         q[0] += 1;
110         r = cut(r * q, 2 * m);
111     }
112     return cut(r, n);
113 }
114
115 Poly power(Poly a, long long k, int n) {
116     if (k == 0) return Poly{1};
117     int shift = 0;
118     while (shift < (int)a.size() && a[shift] == 0) ++shift;
119     if (shift == (int)a.size() || 1LL * shift * k >= n) return Poly(n);
120     mint lead = a[shift];
121     a.erase(a.begin(), a.begin() + shift);
122     for (mint& x : a) x /= lead;
123     Poly r = exponential(logarithm(a, n - shift * k) * mint(k), n - shift * k);
124     r.insert(r.begin(), shift * k, 0);
125     return cut(r * lead.pow(k), n);
126 }
127
128 Poly divide(Poly a, Poly b) { // quotient; b != 0
129     norm(a); norm(b);
130     if (a.size() < b.size()) return {};
131     int n = (int)a.size() - (int)b.size() + 1;
132     reverse(a.begin(), a.end()); reverse(b.begin(), b.end());
133     Poly q = cut(a * inverse(b, n), n);
134     reverse(q.begin(), q.end());
135     return q;
136 }
137 Poly modulo(Poly a, const Poly& b) {
138     Poly q = divide(a, b);
```

```

139 Poly r = a - q * b;
140 r.resize(min((int)r.size(), max(0, (int)b.size() - 1)));
141 norm(r); return r;
142 }

```

6.20 Linear Algebra (Gauss)

Complexity: $\mathcal{O}(n^3)$

Gaussian Elimination: Matrix Determinant, Matrix Rank, System of Linear Equations

```

1 // Complexity:  $\mathcal{O}(N^3)$  or  $\mathcal{O}(N^3 / 64)$  with Bitset
2 // Gaussian Elimination: Determinant, Matrix Inversion, System of
  Linear Equations  $Ax = b$ 
3
4 function gauss(A[N][M], b[N]):
5     // Augmented matrix [A | b]
6     rank = 0, det = 1
7     for col = 0 to M - 1:
8         pivot = find_row_with_max_abs(col, from_row = rank)
9         if abs(A[pivot][col]) < EPS:
10             det = 0; continue
11         swap(row[rank], row[pivot])
12         if rank != pivot: det = -det
13         det *= A[rank][col]
14
15         // Eliminate column entries below and above pivot
16         for r = 0 to N - 1:
17             if r != rank and abs(A[r][col]) > EPS:
18                 factor = A[r][col] / A[rank][col]
19                 row[r] -= factor * row[rank]
20
21         rank += 1
22     return (rank, det, solution = [b[i] / A[i][i] for i in 0..rank
  -1])

```

6.21 XOR Linear Basis

Complexity: $\mathcal{O}(B)$ insert/query ($B \leq 60$)

XOR Linear Basis: Find Max/Min/k-th XOR subset, span of 2^{size} distinct values

```

1 // Complexity:  $\mathcal{O}(B)$  per insert,  $B = 60$  bits
2 // Recognition: Subset XOR max/min/k-th, distinct subset XOR sums,
  cycle XOR in graphs
3
4 struct XORBasis:
5     basis[B] = {0}, sz = 0
6
7     function insert(mask):
8         for i = B - 1 down to 0:
9             if not (mask & (1 << i)): continue
10            if not basis[i]:
11                basis[i] = mask; sz += 1
12                return True
13            mask ^= basis[i]
14        return False // mask is linearly dependent
15
16     function query_max():
17         res = 0
18         for i = B - 1 down to 0:
19             res = max(res, res ^ basis[i])
20        return res
21
22     function query_kth(k):
23         // Rebuild reduced row echelon form first:
24         // for each basis[i], eliminate 1-bits in basis[j] (j > i)

```

```

25     // If 0 is in span (insert failed at least once), decrement
  k if k >= 1
26     res = 0, total_distinct = 1 << sz
27     if k > total_distinct: return -1
28     for i = 0 to sz - 1:
29         if k & (1 << i): res ^= reduced_basis[i]
30     return res

```

6.22 Burnside Lemma & Pólya Theorem

Count distinct equivalence classes under group actions, Necklaces, Bracelets

- $|Classes| = \frac{1}{|G|} \sum_{g \in G} |Fix(g)| = \frac{1}{|G|} \sum_{g \in G} k^{c(g)}$.

- Necklace (n rotations only): $N(n, k) = \frac{1}{n} \sum_{d|n} \phi(d) k^{n/d}$.

- Bracelet (Rotations + Reflections):

- n odd: $B(n, k) = \frac{1}{2} N(n, k) + \frac{1}{2} k^{(n+1)/2}$.

- n even: $B(n, k) = \frac{1}{2} N(n, k) + \frac{1}{4} (k^{n/2} + k^{n/2+1})$.

6.23 Game Strategy & State Reasoning

Complexity: $\mathcal{O}(V + E)$

Winning/Losing state axioms, Retrograde analysis, Symmetry, Parity, Sprague-Grundy MEX, Nim variants

```

1 // GAME REASONING & PLAYBOOK (W/L, MEX, STRATEGIES)
2 // -----
3
4 // 1. STATE CLASSIFICATION (N-pos / P-pos)
5 // - Terminal: Normal play (no move -> LOSE);
6 //           Misère play (no move -> WIN).
7 // - WIN (W / N-pos): EXISTS >= 1 move to LOSE state.
8 // -> Move to LOSE state to force opponent's loss.
9 // - LOSE (L / P-pos): ALL valid moves lead to WIN.
10 // -> Any move leaves opponent in a WIN state.
11 // - DRAW (D): Cycle/loop where neither can force win.
12
13 // 2. DEDUCTION METHODOLOGY (Mental Checklist)
14 // 1. Formalize state: coords, piles, parity, mask.
15 // 2. Small-case brute-force ( $N \leq 20$ ) with DP / MEX.
16 // 3. Print W/L table or Grundy sequence  $g(0), g(1) \dots$ 
17 // 4. Check patterns: Periodicity  $g(n) = g(n \% T)$ ,
18 //    bit parity, trailing zeros, or reduction to Nim.
19 // 5. If independent subgames -> Sprague-Grundy XOR.
20
21 // 3. CORE PROBLEM-SOLVING TACTICS
22 // A. SYMMETRY / MIRRORING:
23 //   - P2 mirrors P1 across center/axis -> P2 wins.
24 //   - P1 makes initial central move leaving symmetric
25 //     halves, then mirrors P2 -> P1 wins.
26 //   - Invariant: After your turn, state is symmetric.
27 // B. INVARIANTS & PARITY:
28 //   - Track I(S) % 2 (sum, components, inversions).
29 //   - If each move flips parity, enforce winning parity.
30 // C. PAIRING STRATEGY:
31 //   - Partition board into pairs (a_i, b_i).
32 //   - Opponent plays a_i -> you immediately play b_i.
33 //   - Perfect matching on moves -> P2 always wins.
34 // D. STRATEGY STEALING:
35 //   - If extra move is never bad, P1 cannot lose.
36
37 // 4. RETROGRADE ANALYSIS (GRAPH WITH CYCLES) -  $\mathcal{O}(V+E)$ 
38 const int DRAW = 0, WIN = 1, LOSE = -1;
39

```

```

40 vector<int> retrograde_analysis(int n, const vector<vector<int>>&
  adj) {
41     vector<int> out_deg(n), res(n, DRAW);
42     vector<vector<int>> radj(n);
43     queue<int> q;
44
45     for (int u = 0; u < n; ++u) {
46         out_deg[u] = adj[u].size();
47         for (int v : adj[u]) radj[v].push_back(u);
48         if (out_deg[u] == 0) {
49             res[u] = LOSE; // Normal play: terminal LOSE
50             q.push(u);
51         }
52     }
53
54     while (!q.empty()) {
55         int u = q.front(); q.pop();
56         for (int p : radj[u]) {
57             if (res[p] != DRAW) continue;
58             if (res[u] == LOSE) {
59                 res[p] = WIN; q.push(p);
60             } else if (res[u] == WIN) {
61                 if (--out_deg[p] == 0) {
62                     res[p] = LOSE; q.push(p);
63                 }
64             }
65         }
66     }
67     return res; // WIN, LOSE, or DRAW (loop)
68 }
69
70 // 5. SPRAGUE-GRUNDY & SUBGAMES
71 // - Independent sum:  $g(G) = g(G_1) \oplus \dots \oplus g(G_k)$ .
72 // -  $g(u) = \text{mex}(\{g(v) : u \rightarrow v\})$ . Win  $\Leftrightarrow g(G) \neq 0$ .
73 // - Tree Hackenbush:  $g(u) = \text{XOR}_{v \in \text{child}(u)} (g(v)+1)$ .
74
75 int calc_mex(const vector<int>& vals) {
76     int m = vals.size();
77     vector<bool> seen(m + 1, false);
78     for (int x : vals) if (x <= m) seen[x] = true;
79     for (int i = 0; i <= m; ++i) if (!seen[i]) return i;
80     return m + 1;
81 }
82
83 int get_grundy(int u, vector<int>& g, const vector<vector<int>>&
  adj) {
84     if (g[u] != -1) return g[u];
85     vector<int> nxt;
86     for (int v : adj[u]) nxt.push_back(get_grundy(v, g, adj));
87     return g[u] = calc_mex(nxt);
88 }
89
90 // 6. CLASSIC NIM VARIANTS
91 // - Nim:  $S = \text{XOR}(a_i)$ . Win  $\Leftrightarrow S \neq 0$ .
92 // - Subtraction(1..k):  $g(a) = a \% (k+1)$ .
93 // - Moore's Nim_k: Lose  $\Leftrightarrow \text{sum}_i \text{bit}_b(a_i) \% (k+1) == 0$ .
94 // - Staircase Nim: Standard Nim on odd stairs (1,3,5...).
95 // - Misère: all  $a_i \leq 1 \rightarrow$  Win  $\Leftrightarrow (\text{count of } 1s) \% 2 == 0$ ;
96 //           exists  $a_i > 1 \rightarrow$  Win  $\Leftrightarrow \text{XOR}(a_i) \neq 0$ .

```

• Winning ($\mathcal{W}$ / N-pos) vs Losing ($\mathcal{L}$ / P-pos) State Axioms:

- **Terminal State:** Normal play (no moves $\implies \mathcal{L}$); Misère play (no moves $\implies \mathcal{W}$).
- **Winning State $\mathcal{W}$:** $\exists v \in \text{next}(u)$ such that $v \in \mathcal{L}$ (at least ONE move leads to an $\mathcal{L}$ -state).
- **Losing State $\mathcal{L}$:** $\forall v \in \text{next}(u)$, $v \in \mathcal{W}$ (ALL moves lead to $\mathcal{W}$ -states; or no moves exist).

- **Optimal Play:** From $\mathcal{W}$, always choose a move to $\mathcal{L}$. Opponent from $\mathcal{L}$ is forced to move back to $\mathcal{W}$.

• Problem-Solving Deduction Workflow:

- **Retrograde BFS** ($O(V + E)$): Propagate outcomes backward from terminal states using out-degree counters; unvisited states in cycles are **DRAW**.
- **Symmetry / Mirroring:** If P2 can mirror P1's moves across center/axes, P2 wins. If P1 can make a central move leaving symmetric halves, P1 wins.
- **Invariants & Parity:** Track $I(S) \bmod 2$ (sum, components, inversions). Control parity to force terminal state on opponent.
- **Pairing Strategy:** Partition board into disjoint matched pairs (a_i, b_i) . When P1 takes a_i , P2 immediately takes b_i .
- **Small-case Pattern Finding:** Simulate for $N \leq 20$, compute SG values $g(n)$, find periodicity $g(n) = g(n \bmod T)$ or bit conditions.

• Sprague-Grundy Theorem (Independent Subgames):

- $g(G) = \bigoplus g(G_i)$, where $g(u) = \text{mex}(\{g(v) \mid u \rightarrow v\})$. Win $\iff g(G) \neq 0$.
- **Tree Hackenbush (Colon Principle):** Branch node $g(u) = \bigoplus_{v \in \text{children}} (g(v) + 1)$.

• Classic Nim Variations:

- **Standard Nim:** XOR sum $S = \bigoplus a_i$. Win $\iff S \neq 0$.
- **Bounded Subtraction Nim** (take 1..k): $g(a) = a \bmod (k + 1) \implies \text{Win} \iff \bigoplus (a_i \bmod (k + 1)) \neq 0$.
- **Moore's Nim_k** (take from $\leq k$ piles): Lose $\iff \forall \text{ bit position } b, \sum_i \text{bit}_b(a_i) \equiv 0 \pmod{k + 1}$.
- **Staircase Nim:** Equivalent to standard Nim on odd-indexed stairs $(1, 3, 5, \dots)$.
- **Misère Nim** (last move LOSES):
 - * All piles ≤ 1 : Win $\iff$ count of size-1 piles is EVEN ($S = 0$).
 - * At least one pile > 1 : Identical to standard Nim (Win $\iff S \neq 0$).

6.24 Probability & Expected Value

Linearity of expectation, Indicator variables, Coupon collector, Inequalities

- **Linearity of Expectation:** $E[X_1 + \dots + X_n] = E[X_1] + \dots + E[X_n]$ ($\forall X_i$).
- **Indicator Variables:** $E[X] = \sum P(A_i)$.
- **Geometric Distribution:** $E[X] = 1/p$, $\text{Var}(X) = (1 - p)/p^2$.
- **Coupon Collector:** $E[T] = n \sum_{i=1}^n \frac{1}{i} = nH_n \approx n(\ln n + \gamma)$ ($\gamma \approx 0.5772$).
- **Markov:** $P(X \geq a) \leq \frac{E[X]}{a}$; **Chebyshev:** $P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$.

6.25 Bignum

Complexity: $O(n^2)$
Extreme Bignum Template

```
1 using bignum = vector<int>;
2
3 bignum operator + (const bignum &a, const bignum &b){
4     bignum res;
5     int carry = 0;
6     for(int i = 0; i < max(b.size(), a.size()); i++){
7         int c1 = (i < sz(a)) ? a[i] : 0;
8         int c2 = (i < sz(b)) ? b[i] : 0;
9         int c = (c1 + c2 + carry);
10        res.pb(c % 10);
11        carry = c / 10;
12    }
13    while(carry) {
14        res.pb(carry % 10);
15        carry /= 10;
16    }
17    return res;
18 }
19 bignum operator * (const bignum &a, int k){
20     bignum res;
21     int carry = 0;
22     for(int i = 0; i < a.size(); i++){
23         int c = (a[i] * k + carry);
24         res.pb(c % 10);
25         carry = c / 10;
26     }
27     while(carry) {
28         res.pb(carry % 10);
29         carry /= 10;
30     }
31     return res;
32 }
```

7 Geometry

7.1 Primitive Geometry

Complexity: $O(1)$ per operation
Core 2D primitives: Points, Vectors, Orientation, Line Intersections, Projections

```
1 // Complexity: O(1) per primitive operation
2 // Core 2D Geometry: Points, Vectors, Lines, Circles
3
4 struct Point:
5     x, y
6     operator + (p) => Point(x + p.x, y + p.y)
7     operator - (p) => Point(x - p.x, y - p.y)
8     operator * (k) => Point(x * k, y * k)
9     dot(p) => x * p.x + y * p.y
10    cross(p) => x * p.y - y * p.x // > 0 if p is counter-clockwise
11                                     (left turn)
12    dist2() => x * x + y * y
13    dist() => sqrt(dist2())
14
15 function orientation(a, b, c):
16     val = (b - a).cross(c - a)
17     if abs(val) < EPS: return 0 // Collinear
18     return 1 if val > 0 else -1 // 1: CCW (Left), -1: CW (Right)
19
20 function line_intersection(p1, p2, q1, q2):
21     // Line 1 through p1, p2; Line 2 through q1, q2
22     cross_val = (p2 - p1).cross(q2 - q1)
23     if abs(cross_val) < EPS: return NULL // Parallel or coincident
24     t = (q1 - p1).cross(q2 - q1) / cross_val
25     return p1 + (p2 - p1) * t
```

7.2 Convex Hull

Complexity: $O(n \log n)$

Monotone Chain / Graham Scan: Smallest convex polygon enclosing point set

```
1 // Complexity: O(N log N) time, O(N) space
2 // Monotone Chain algorithm: Constructs convex hull in counter-
3 // clockwise order
4
5 function convex_hull(points P):
6     sort(P) // Lexicographically by x, then y
7     lower = [], upper = []
8
9     for p in P:
10        while len(lower) >= 2 and orientation(lower[len-2], lower[
11            len-1], p) <= 0:
12            lower.pop()
13        lower.append(p)
14
15    for p in reversed(P):
16        while len(upper) >= 2 and orientation(upper[len-2], upper[
17            len-1], p) <= 0:
18            upper.pop()
19        upper.append(p)
20
21    lower.pop(); upper.pop()
22    return lower + upper
```

7.3 Rotating Calipers

Complexity: $O(n)$

Rotating Calipers: Polygon diameter (farthest pair), Minimum bounding rectangle

```
1 // Complexity: O(N) on convex hull in CCW order
2 // Recognition: Maximum pairwise distance (diameter), Minimum
3 // bounding box, Max triangle area
4
5 function convex_diameter(poly CH):
6     n = len(CH)
7     if n <= 1: return 0
8     if n == 2: return dist(CH[0], CH[1])
9
10    max_d2 = 0, k = 1
11    // Find initial farthest point k from edge (CH[0], CH[1])
12    while triangle_area(CH[n-1], CH[0], CH[(k + 1) % n]) >
13        triangle_area(CH[n-1], CH[0], CH[k]):
14        k = (k + 1) % n
15
16    for i = 0 to n - 1:
17        // Advance caliper pointer k while distance to edge (CH[i],
18        // CH[i+1]) increases
19        next_i = (i + 1) % n
20        while triangle_area(CH[i], CH[next_i], CH[(k + 1) % n]) >=
21            triangle_area(CH[i], CH[next_i], CH[k]):
22            k = (k + 1) % n
23        max_d2 = max(max_d2, dist2(CH[i], CH[k]))
24        max_d2 = max(max_d2, dist2(CH[next_i], CH[k]))
25    return sqrt(max_d2)
```

7.4 Minkowski Sum

Complexity: $\mathcal{O}(|P| + |Q|)$

Minkowski Sum of two convex polygons. Min polygon distance:
 $\text{dist}(P, Q) = \text{dist}(O, P - Q)$

```
1 // Complexity:  $\mathcal{O}(|P| + |Q|)$  time
2 // Computes Minkowski Sum  $P + Q = \{p + q \mid p \in P, q \in Q\}$  for
   convex polygons in CCW order
3
4 function minkowski_sum(poly P, poly Q):
5     // Reorder vertices starting at bottom-left
6     rotate_to_min_vertex(P); rotate_to_min_vertex(Q)
7     P.append(P[0]); P.append(P[1])
8     Q.append(Q[0]); Q.append(Q[1])
9
10    i = 0, j = 0, result = []
11    while i < len(P) - 2 or j < len(Q) - 2:
12        result.append(P[i] + Q[j])
13        cross_val = (P[i+1] - P[i]).cross(Q[j+1] - Q[j])
14        if cross_val >= 0 and i < len(P) - 2: i += 1
15        if cross_val <= 0 and j < len(Q) - 2: j += 1
16    return result
```

7.5 Point in Convex Polygon

Complexity: $\mathcal{O}(\log n)$

Test if point lies inside convex polygon via polar angle binary search from P_0

```
1 // Complexity:  $\mathcal{O}(\log N)$  time per query on convex polygon in CCW
   order
2 // Recognition: Fast point inclusion testing in convex polygon
3
4 function point_in_convex_polygon(poly P, Point pt):
5     n = len(P)
6     // Check if pt is within the sector between  $P[0] \rightarrow P[1]$  and  $P[0] \rightarrow P[n-1]$ 
7     if orientation(P[0], P[1], pt) < 0 or orientation(P[0], P[n-1], pt) > 0:
8         return OUTSIDE
9
10    // Binary search to find triangle  $P[0] - P[mid] - P[mid+1]$ 
       containing pt
11    low = 1, high = n - 2
12    while low <= high:
13        mid = (low + high) / 2
14        if orientation(P[0], P[mid], pt) >= 0:
15            pos = mid; low = mid + 1
16        else:
17            high = mid - 1
18
19    // Check if pt is inside triangle  $P[0] - P[pos] - P[pos+1]$ 
20    val = orientation(P[pos], P[pos+1], pt)
21    if val > 0: return INSIDE
22    if val == 0: return ON_BOUNDARY
23    return OUTSIDE
```

7.6 Line Hull Intersection

Complexity: $\mathcal{O}(\log n)$

Binary search extreme vertices along normal vector to find intersection points

```
1 // Complexity:  $\mathcal{O}(\log N)$  time per line query
2 // Finds intersection of line L with convex hull CH using binary
   search
3
```

```
4 function extreme_vertex(poly CH, direction_vector V):
5     // Binary search on ternary search of dot product  $(P[i] \cdot V)$ 
6     low = 0, high = len(CH) - 1
7     while high - low > 2:
8         m1 = low + (high - low) / 3
9         m2 = high - (high - low) / 3
10        if CH[m1].dot(V) < CH[m2].dot(V): low = m1
11        else: high = m2
12    return argmax_{i in [low..high]} (CH[i].dot(V))
13
14 function line_hull_intersect(poly CH, Line L):
15     n1 = extreme_vertex(CH, L.normal)
16     n2 = extreme_vertex(CH, -L.normal)
17     if L.side(CH[n1]) * L.side(CH[n2]) > 0:
18         return (intersects = False, [])
19     // Binary search on both chains between n1 and n2 to find exact
       crossing edges
20    return (intersects = True, [binary_search_cross(CH, n1, n2, L),
        binary_search_cross(CH, n2, n1, L)])
```

7.7 Polygon Raycast

Complexity: $\mathcal{O}(n)$

Horizontal raycasting point-in-polygon test for arbitrary simple polygons

```
1 // Complexity:  $\mathcal{O}(N)$  time for point in polygon,  $\mathcal{O}(\log N)$  for convex
   polygon
2 // Ray-Casting algorithm: Cast horizontal ray from P to +infinity,
   count edge crossings
3
4 function point_in_polygon(poly P, Point pt):
5     crossings = 0, n = len(P)
6     for i = 0 to n - 1:
7         a = P[i], b = P[(i + 1) % n]
8         if on_segment(a, b, pt): return ON_BOUNDARY
9         if (a.y > pt.y) != (b.y > pt.y):
10            // Calculate x-coordinate of intersection with
               horizontal line  $y = pt.y$ 
11            x_cross = a.x + (pt.y - a.y) * (b.x - a.x) / (b.y - a.y)
12            if pt.x < x_cross:
13                crossings += 1
14    return INSIDE if (crossings % 2 == 1) else OUTSIDE
```

7.8 Half-plane Intersection

Complexity: $\mathcal{O}(n \log n)$

Intersection of half-planes forming convex polygon via Deque

```
1 // Complexity:  $\mathcal{O}(N \log N)$  time,  $\mathcal{O}(N)$  space
2 // Computes convex polygon formed by intersection of halfplanes (
   directed lines with valid side on left)
3
4 struct Halfplane:
5     p, pq // point p and direction vector pq
6
7 function halfplane_intersection(halfplanes H):
8     sort(H, by polar_angle)
9     deque<Halfplane> dq; deque<Point> pts
10
11    for h in H:
12        while len(pts) > 0 and not h.contains(pts.back()):
13            pts.pop_back(); dq.pop_back()
14        while len(pts) > 0 and not h.contains(pts.front()):
15            pts.pop_front(); dq.pop_front()
16        if len(dq) > 0:
```

```
17            pts.push_back(line_intersection(dq.back(), h))
18            dq.push_back(h)
19
20    while len(pts) > 0 and not dq.front().contains(pts.back()):
21        pts.pop_back(); dq.pop_back()
22    if len(dq) < 3: return [] // Empty intersection
23    pts.push_back(line_intersection(dq.back(), dq.front()))
24    return pts // Vertices of convex polygon
```

7.9 Smallest Enclosing Circle

Complexity: $\mathcal{O}(n)$

Welzl's randomized algorithm for minimum bounding circle

```
1 // Complexity:  $\mathcal{O}(N)$  expected time
2 // Welzl's Algorithm: Smallest enclosing disk containing all points
   P
3
4 function circle_from_1_or_2_or_3_points(R):
5     if len(R) == 0: return Circle(origin, 0)
6     if len(R) == 1: return Circle(R[0], 0)
7     if len(R) == 2: return Circle((R[0] + R[1]) / 2, dist(R[0], R[1]) / 2)
8     return circumcircle(R[0], R[1], R[2])
9
10 function welzl(points P, boundary R, n):
11     if n == 0 or len(R) == 3:
12         return circle_from_1_or_2_or_3_points(R)
13     p = P[n - 1]
14     c = welzl(P, R, n - 1)
15     if c.contains(p): return c
16     return welzl(P, R + [p], n - 1)
17
18 function smallest_enclosing_circle(points P):
19     shuffle(P)
20     return welzl(P, [], len(P))
```

7.10 Circle Tangents

Complexity: $\mathcal{O}(1)$

Determine up to 4 common tangent lines between 2 circles

```
1 // Complexity:  $\mathcal{O}(1)$  time
2 // Common tangents between two circles  $c1(01, r1)$  and  $c2(02, r2)$ 
3
4 struct Line:
5     a, b, c //  $ax + by + c = 0$ 
6
7 function common_tangents(Circle c1, Circle c2):
8     d = dist(c1.0, c2.0), lines = []
9     if d < EPS and abs(c1.r - c2.r) < EPS: return INF // Infinite
       tangents
10
11    // External tangents ( $r1 - r2$ )
12    for sign1 in [1, -1]:
13        // Angle alpha between  $O1 \rightarrow O2$  and tangent
14        cos_val = (c1.r - c2.r * sign1) / d
15        if abs(cos_val) <= 1:
16            sin_val = sqrt(max(0.0, 1 - cos_val * cos_val))
17            for sign2 in [1, -1]:
18                // Construct tangent lines
19                tangent = build_tangent_line(c1, c2, cos_val,
20                    sin_val * sign2)
21                lines.append(tangent)
22    return lines
```

7.11 Geometry Formulas

Planar theorems, circles, triangle centers, circle inversion & 3D primitives

- **Pick's Theorem:** $S = I + \frac{B}{2} - 1 \implies I = S - \frac{B}{2} + 1$, where $B = \sum \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|)$.
- **Euler's Planar Formula:** $V - E + F = 1 + C$ ($V - E + F = 2$ for connected planar graphs).
- **Ptolemy's Theorem:** Cyclic quad $ABCD \iff AC \cdot BD = AB \cdot CD + BC \cdot AD$.
- **Stewart's Theorem:** $b^2m + c^2n = a(d^2 + mn)$ (cevia d divides side a into m, n).
- **Bretschneider's Formula** (General quadrilateral area):

$$S = \sqrt{(p-a)(p-b)(p-c)(p-d) - abcd \cos^2 \left(\frac{A+C}{2} \right)}$$

(Cyclic quad $\implies \cos(\frac{A+C}{2}) = 0$, reduces to Brahmagupta's formula).

- **Descartes' Theorem** (4 mutually tangent circles):

$$\left(\sum_{i=1}^4 k_i \right)^2 = 2 \sum_{i=1}^4 k_i^2 \quad \left(k_i = \pm \frac{1}{R_i} \right)$$

- **Circle Inversion:** Center O , radius R . $P \mapsto P'$ satisfies $OP \cdot OP' = R^2$. Maps lines not through $O \longleftrightarrow$ circles through O ; preserves angles and tangencies.
- **Circle-Circle Intersection:** Let $d = |O_1 - O_2|$. Intersects when $|R_1 - R_2| \leq d \leq R_1 + R_2$, angle $\alpha = \arccos \left(\frac{R_1^2 + d^2 - R_2^2}{2R_1d} \right)$.
- **Apollonius Circle:** Locus $\frac{PA}{PB} = k$ ($k \neq 1$) is a circle with diameter connecting the internal and external division points of AB with ratio k .
- **Circumcenter O & Orthocenter H :** Let $d_B = x_B^2 + y_B^2 - x_A^2 - y_A^2$, $d_C = x_C^2 + y_C^2 - x_A^2 - y_A^2$:

$$x_O = \frac{d_B(y_C - y_A) - d_C(y_B - y_A)}{D}$$

$$y_O = \frac{(x_B - x_A)d_C - (x_C - x_A)d_B}{D}$$

$$D = 2[(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)]$$

$$H = A + B + C - 2O$$

- **3D Primitives:**

- 3D Cross Product: $\vec{A} \times \vec{B} = (A_y B_z - A_z B_y, A_z B_x - A_x B_z, A_x B_y - A_y B_x)$.
- Tetrahedron Volume: $V = \frac{1}{6} |((B - A) \times (C - A)) \cdot (D - A)|$.
- Plane through P_0 with normal $\vec{n}$: $\vec{n} \cdot (P - P_0) = 0$; Point-plane distance: $\frac{|\vec{n} \cdot (P - P_0)|}{|\vec{n}|}$.

7.12 AREA

Complexity: $\mathcal{O}(n \log n)$

Area of Union of Rectangles with Sweep-line and Segment Tree

```

1 // Complexity:  $\mathcal{O}(N \log N)$  time
2 // Sweep-line with Segment Tree to compute Area of Union of Axis-
  Aligned Rectangles
3
4 struct Event:
5     x, y1, y2, type (+1 for start, -1 for end)
6
7 function union_area_rectangles(rectangles):
8     events = [], Y_coors = []
9     for r in rectangles:
10         events.append(Event(r.x1, r.y1, r.y2, +1))
11         events.append(Event(r.x2, r.y1, r.y2, -1))
12         Y_coors.append(r.y1); Y_coors.append(r.y2)
13     sort(events, by x)
14     unique_sorted(Y_coors)
15
16     // Segment tree maintains covered length on Y intervals
17     segtree_init(Y_coors)
18     total_area = 0, last_x = events[0].x
19
20     for e in events:
21         total_area += segtree.total_covered_length() * (e.x -
          last_x)
22         segtree.update_range(e.y1, e.y2, e.type)
23         last_x = e.x
24     return total_area

```

7.13 Manhattan MST

Complexity: $\mathcal{O}(n \log n)$

Manhattan Minimum Spanning Tree via 8-octant sweep

```

1 // Complexity:  $\mathcal{O}(N \log N)$  time
2 // Minimum Spanning Tree under Manhattan distance: generates  $\mathcal{O}(N)$ 
  candidate edges via 8 octants
3
4 function manhattan_mst_edges(points P):
5     candidate_edges = []
6     for dir1 = 0 to 1:
7         for dir2 = 0 to 1:
8             for dir3 = 0 to 1:
9                 sort(P, by x + y)
10                // Use Fenwick tree / map to find best point
11                minimizing  $(x_j - y_j)$  with  $y_j \leq y_i$ 
12                BIT_sweep_closest(P, candidate_edges)
13                for p in P: swap(p.x, p.y)
14                for p in P: p.x = -p.x
15                for p in P: p.y = -p.y
16            return kruskal_mst(P, candidate_edges)

```

7.14 Planar Dual Graph

Complexity: $\mathcal{O}(E \log E)$

Planar embedding to dual graph: extracts faces (including outer face) and dual edges via angular sorting on half-edges (darts).

```

1 // Complexity:  $\mathcal{O}(E \log E)$  time,  $\mathcal{O}(V + E)$  space
2 // Constructs planar dual: extracts faces (including outer face)
  and dual edges
3
4 struct PlanarDual:
5     face_count = 0, dual_edges = []
6
7     function build(points P, edges E):
8         // 1. Build directed half-edges (darts)
9         for i, (u, v) in enumerate(E):
10             darts[u].push(2 * i) // u -> v

```

```

11         darts[v].push(2 * i + 1) // v -> u
12
13         // 2. Sort outgoing darts CCW by angle around each vertex
14         for u in 0..V-1:
15             sort(darts[u], by CCW_angle_around(u))
16             for idx, d in enumerate(darts[u]): pos[d] = idx
17
18         // 3. For each dart d = (u -> v), next dart along face
19         boundary is:
20         // the dart immediately clockwise from reverse dart (v
21         -> u)
22         for d in 0..2*E-1:
23             v = to[d], rev_pos = pos[d ^ 1], deg = len(darts[v])
24             next_dart[d] = darts[v][(rev_pos - 1 + deg) % deg]
25
26         // 4. Trace faces along next_dart cycles
27         for start in 0..2*E-1:
28             if face_of[start] != -1: continue
29             cur = start; face_id = face_count++
30             do:
31                 face_of[cur] = face_id
32                 cur = next_dart[cur]
33             while cur != start
34
35         // 5. Dual edge for original edge i is (face_of[2*i],
36         face_of[2*i+1])
37         for i in 0..E-1:
38             dual_edges.push({face_of[2 * i], face_of[2 * i + 1]})

```

8 Misc

8.1 Template

```

1 #include <bits/stdc++.h>
2 #define FOR(i, a, b) for (int i = a; i <= b; i++)
3 #define FORD(i, a, b) for (int i = a; i >= b; i--)
4 #define endl '\n'
5 #define pb push_back
6 #define pf push_front
7 #define fastio ios_base::sync_with_stdio(NULL); cin.tie(NULL); cout
  .tie(NULL);
8
9 //Data structure
10 #define vi vector<int>
11 #define int long long
12 #define ii pair<int, int>
13 #define iii pair<int, ii>
14
15 //Bitmask (absolutely have a () outside of every thing)
16 #define LSB(x) (x & (-x))
17 #define ON(x, i) ((x >> i) & 1)
18 #define OFF(x, i) !ON(x, i)
19 #define SET(x, i) (x | (1LL << i))
20 #define UNSET(x, i) (SET(x, i) ^ (1LL << i))
21
22 #define TASKNAME "test"
23
24 template<typename T> bool maximize(T &a, T b){ if (a < b){ a = b
  ; return true; } return false; }
25 template<typename T> bool minimize(T &a, T b){ if (a > b){ a = b
  ; return true; } return false; }
26 using namespace std;
27
28 const int MOD = 1e9 + 7;
29 void add(int &a, int b){
30     a += b;

```

```

31     if (a >= MOD) a -= MOD;
32 }
33
34 void sub(int &a, int b){
35     a -= b;
36     if (a < 0) a += MOD;
37 }
38 void solve(){
39
40 }
41 main(){
42     fastio;
43     if (fopen(TASKNAME".inp","r")){
44         freopen(TASKNAME".inp","r",stdin);
45         freopen(TASKNAME".out","w",stdout);
46     }
47     int t = 1;
48     //cin >> t;
49     while(t--){
50         solve();
51     }
52 }

```

8.2 Debugging Magic

```

1 #define what_is(x) cerr << #x << " = " << x << endl;
2 #define deb(...) logger(#__VA_ARGS__, __VA_ARGS__)
3 template<typename ...Args>
4 void logger(string vars, Args&&... values) {
5     cout << "{" << vars << " } = {";
6     string delim = "";
7     (... , (cout << delim << values << "}", delim = ", "));
8 }

```

8.3 Expression Parsing

```

1 // Complexity: O(N) time and space
2 // Shunting-Yard algorithm: Parses infix arithmetic expressions
   with operator precedence
3
4 function parse_expression(string expr):
5     stack<double> values; stack<char> ops
6
7     function apply_op():
8         op = ops.pop(); b = values.pop(); a = values.pop()
9         values.push(eval_op(a, b, op))
10
11     for token in tokenize(expr):
12         if is_number(token):
13             values.push(token)
14         else if token == '(':
15             ops.push('(')
16         else if token == ')':
17             while ops.top() != '(': apply_op()
18             ops.pop() // remove '('
19         else: // Operator +, -, *, /, ^
20             while not ops.empty() and precedence(ops.top()) >=
precedence(token):
21                 apply_op()
22             ops.push(token)
23
24     while not ops.empty(): apply_op()
25     return values.top()

```

8.4 Johnson Scheduling

Scheduling on two machines

```

1 // Complexity: O(N log N) time
2 // Johnson's Rule: Optimal scheduling of N jobs on 2 machines in
   sequence (Machine 1 then Machine 2)
3
4 struct Job:
5     id, a (time on M1), b (time on M2)
6
7 function johnson_scheduling(jobs):
8     // Partition jobs into Group 1 (a_i <= b_i) and Group 2 (a_i >
b_i)
9     G1 = [j for j in jobs if j.a <= j.b]
10    G2 = [j for j in jobs if j.a > j.b]
11
12    sort(G1, ascending by a)
13    sort(G2, descending by b)
14
15    return G1 + G2 // Optimal processing order

```

8.5 Matrix Multiplication Check

Check if $C = AB$:

- Generate a $N \times 1$ vector v .
- Calculate $P = ABv - Cv$.
- if $(P \neq 0)$ return false, else the error is $1/2^k$

8.6 Calendar

```

1 // Complexity: O(1) time
2 // Zeller's Congruence: Computes day of the week (0 = Sunday, 1 =
Monday, ..., 6 = Saturday)
3
4 function day_of_week(year, month, day):
5     if month < 3:
6         month += 12; year -= 1
7     K = year % 100, J = year / 100
8     // Zeller's formula for Gregorian calendar
9     h = (day + (13 * (month + 1)) / 5 + K + K / 4 + J / 4 + 5 * J)
% 7
10    return (h + 6) % 7 // Shift so 0 = Sunday

```

8.7 Josephus

```

1 // Complexity: O(N) for linear, O(K log N) for large N, small K
2 // Josephus Problem: 0-indexed survivor among N people eliminating
   every K-th person
3
4 function josephus_linear(N, K):
5     ans = 0
6     for i = 2 to N:
7         ans = (ans + K) % i
8     return ans
9
10 function josephus_fast(N, K):
11     if N == 1: return 0
12     if K == 1: return N - 1
13     if K > N: return (josephus_fast(N - 1, K) + K) % N
14     cnt = N / K
15     res = josephus_fast(N - cnt, K)
16     res -= N % K
17     if res < 0: res += N
18     else: res += res / (K - 1)
19     return res

```

8.8 Interactive Tricks

```

1 // INTERACTIVE CHECKLIST
2 // 1. Read the protocol literally: query format, answer format,
   query limit,
3 // indexing, whether the hidden object may change, and all
   special replies.
4 // 2. Print a query, then flush immediately: cout << "? ..." <<
endl;
5 // (or cout << "? ...\\n" << flush). Never print debug output to
stdout.
6 // 3. If judge returns -1 / an invalid reply: terminate immediately
.
7 // 4. Keep a local query counter and assert it does not exceed the
budget.
8 // 5. Print the final answer with the required prefix, flush, then
return.
9
10 // MONOTONE PREDICATE -> BINARY SEARCH
11 // If ask(x) changes only once (false,false,...,true,true,...),
   find first true:
12 lo = L, hi = R + 1
13 while lo + 1 < hi:
14     mid = (lo + hi) / 2
15     if ask(mid): hi = mid
16     else: lo = mid
17 // Invariants: ask(lo) = false, ask(hi) = true (use sentinels if
necessary).
18 // Variants: find boundary/maximum feasible/minimum feasible;
   exponential search
19 // first if R is unknown. Prove monotonicity before applying this.
20
21 // COMPARE / MAXIMUM / SECOND MAXIMUM
22 // A tournament needs n-1 comparisons to find max. Store, for each
   winner,
23 // every opponent it beat. The second maximum is max among max's
   opponents:
24 // additional ceil(log2 n)-1 comparisons for a balanced tournament.
25 // If query compares ranges, first use it to identify the range
   containing the
26 // answer, then recurse only there; do not repeatedly query
   overlapping ranges.
27
28 // BITWISE RECONSTRUCTION
29 // If a query reveals whether bit b of the answer is 1, test all
   bits directly.
30 // If a query returns AND/OR/XOR of chosen indices, derive missing
   information:
31 // x + y = (x AND y) + (x OR y)
32 // x XOR y = (x OR y) - (x AND y)
33 // Query a few anchors, solve their values algebraically, then
   recover every
34 // other value relative to an anchor. Count equations vs unknown
   values first.
35
36 // SPLIT / GROUP TEST
37 // A yes/no query on a set often gives a group-testing decision
   tree:
38 // split candidates into halves; query one half; keep the half
   consistent with
39 // the reply. Target O(log N) depth. For "one lie" / noisy
   replies, repeat
40 // independently or maintain all candidates consistent with <= k
   lies.
41
42 // PERMUTATION / PARTNER RECONSTRUCTION
43 // Treat each answer as a constraint, not a value. Maintain
   feasible positions
44 // or a bipartite graph of possible pairs; use queries that
   eliminate about half
45 // the remaining possibilities. Once only matching ambiguity

```

```

    remains, solve it
46 // offline (matching/2-SAT), then query only to disambiguate if
    protocol allows.
47
48 // RANDOMIZED SAMPLING
49 // When a query samples/checks a random location, calculate success
    probability.
50 // Missing a bad set of size k after t uniform samples from n is <=
    (1-k/n)^t.
51 // Fix an RNG seed only for local reproducibility; never rely on
    luck when the
52 // remaining failure probability is not far below the judge's
    tolerance.
53
54 // FINAL SANITY: query is legal, indices distinct if required, no
    empty set,
55 // correct 0/1-indexing, exactly one final answer, and every output
    is flushed.

```

8.9 Random, Bit Tricks, Bitset

```

1 // Complexity: O(1) or O(3^N) for submask iteration
2 // Useful Bit Hacks, Bitset, and Submask Iterations
3
4 // 1. Iterate all submasks of a given mask:
5 for sub = mask; sub > 0; sub = (sub - 1) & mask:
6     process(sub)
7 process(0)
8
9 // 2. Gosper's Hack: Iterate all subsets of size K from N elements
10 function gopers_hack(K, N):
11     mask = (1 << K) - 1
12     while mask < (1 << N):
13         process(mask)
14         c = mask & -mask, r = mask + c
15         mask = ((r ^ mask) >> 2) / c | r
16
17 // 3. Bit Tricks:
18 // Lowest set bit: x & -x
19 // Clear lowest set bit: x & (x - 1)
20 // Builtin fast operations:
21 // __builtin_clz(x) (leading zeros), __builtin_ctz(x) (trailing
    zeros), __builtin_popcountll(x)
22
23 // 4. Bitmask DP State Reduction (Eliminate step/size loop):
24 // In matching/permutations: step i is implicitly
    __builtin_popcount(mask).
25 // dp[i][mask] -> dp[mask], reducing space from O(N * 2^N) to O(2^N
    ).
26 // Iterating mask = 0..(1<<N)-1 guarantees topological DAG order
    without outer loop for i.

```

8.10 Transformations

- **When Stuck:** Re-read statement! Formulate small cases ($N \leq 6$) to find patterns/invariants; deduce complexity ($N \leq 20 \rightarrow 2^N, 500 \rightarrow$

$N^3, 5000 \rightarrow N^2, 2 \cdot 10^5 \rightarrow N \log N$).

- **Opt $\rightarrow$ Decision (Binary Search):** If predicate $P(X)$ is monotonic, binary search over X to solve feasibility.
- **Global Permutation $\rightarrow$ Local Swap:** Greedy exchange argument: swapping $(i, j) \rightarrow (j, i)$ improves objective $\iff f(i, j) < f(j, i)$.
- **Constraints $\rightarrow$ Graph / Flow / Cut:** 2-SAT, difference constraints ($x_j - x_i \leq w$), Project Selection (Min-Cut), Hall's Marriage, Dilworth (DAG path cover = $|V| - \text{MaxMatching}$).
- **Algebraic $\rightarrow$ Geometry / Convexity:** Upper/lower envelopes, Convex Hull, Slope Trick, Minkowski sum, CHT.
- **Global Range $\rightarrow$ Divide & Conquer:** Split $[l, r] \rightarrow [l, m]$ and $[m + 1, r]$ + cross contributions (CDQ D&C, D&C DP).
- **Offline Deletions $\rightarrow$ Additions (Time Reversal):** Invert query timeline $Q \rightarrow 1$ to turn deletions into DSU additions. Game DP: work backward from terminal state.
- **Search Space Split (Meet-in-Middle):** Split $N = N/2 + N/2$, reducing $2^N \rightarrow 2^{N/2}$ via hash table / two pointers.
- **Exact ($= k$) $\rightarrow$ Lagrange Penalty ($\geq k$):** Alien's Trick (WQS) relaxes exact k on convex DP; Binomial Inversion / PIE converts exact k to $\geq k$.
- **Total Evaluation $\rightarrow$ Element Contribution:** Invert perspective: sum contribution of each element/edge/pair (u, v) over all valid states (Fubini).
- **Threshold Partitioning ($\sqrt{N}$ Heavy/Light):** Elements $\leq B$ updated statically/cached; elements $> B$ (at most N/B) processed dynamically (Mo's algorithm).
- **State Compression:** In DP, step count $i = \text{popcount}(\text{mask}) \implies dp[i][\text{mask}] \rightarrow dp[\text{mask}]$, preserving topological order.
- **Exact Count $\rightarrow$ Complementary / PIE:** Count valid states via Total - Invalid, Stars & Bars, PIE.
- **Impartial Game $\rightarrow$ Nim Sum:** Transform games on DAG into XOR sum of Sprague-Grundy values ($SG(u) = \text{mex}\{SG(v)\}$).
- **General Case $\rightarrow$ Simplified Special Case:** Solve easier restrictions first ($A_i \in \{0, 1\}$, tree, $N \leq 6$) to reveal pattern before generalizing.
- **Exhaustive Search $\rightarrow$ Branch & Bound:** Prune DFS subtrees with admissible bounding heuristics.
- **Manhattan $\leftrightarrow$ Chebyshev (45° Rotation):** $(x, y) \mapsto (x + y, x - y) \implies |x_1 - x_2| + |y_1 - y_2| = \max(|x'_1 - x'_2|, |y'_1 - y'_2|)$. Decouples 2D dimensions into independent 1D axes!
- **Max/Min & Absolute Value Decomp:** $\max(A, B) = \frac{A+B+|A-B|}{2}$, $\min(A, B) = \frac{A+B-|A-B|}{2}$; $|x - a| = \max(x - a, a - x)$. To minimize $\sum |x - a_i| + |y - b_i|$, split by 4 sign pairs $(\pm x \pm y)$.
- **Threshold / Layer Counting (Fubini):** $\sum_{i=1}^n A_i = \sum_{v=1}^{\max A} \text{count}(A_i \geq v)$ (Summing values $\longleftrightarrow$ Counting elements $\geq v$).
- **Range Add $\rightarrow$ Difference Array / Prefix Sums:** $A_i = \sum_{j=1}^i D_j$ ($D_i = A_i - A_{i-1}$); Range add $[l, r] \rightarrow 2$ point updates on D . Static sums $\rightarrow S_r - S_{l-1}$.

- **Range / All-Pairs $\rightarrow$ Adjacent Condition:** $\min_{i \neq j} |A_i - A_j| = \min_i (A_{i+1} - A_i)$ after sort; $A_j - A_i \geq (j - i)D \iff A'_{i+1} \geq A'_i$ ($A'_i = A_i - iD$); Majority ($> 50\%$) subsegment ≥ 2 always contains length 2 (x, x) or 3 (x, y, x) witness.

- **Product to Sum:** $ab = \binom{a+b}{2} - \binom{a}{2} - \binom{b}{2}$; $\prod A_i \rightarrow \sum \log A_i$ (or log-probabilities).

- **Bitwise Identities:** $A + B = (A \oplus B) + 2(A \text{ AND } B) = (A \text{ OR } B) + (A \text{ AND } B)$; $A \oplus B = (A \text{ OR } B) - (A \text{ AND } B)$; Adjacent bit flips: $\sum_{i=1}^n \text{popcount}((i - 1) \oplus i) = 2n - \text{popcount}(n)$.

- **Set / Unique-Permutation $\rightarrow$ XOR Hashing:** Assign random 64-bit r_v to value v . XOR is mergeable under symmetric difference; for general multisets use additive random signatures (XOR only keeps multiplicity modulo 2).

- **Equivalence $\rightarrow$ Signature:** Sequence $\rightarrow$ rolling hash; tree $\rightarrow$ sorted child hashes; cyclic object $\rightarrow$ canonicalize first. For multiset M : assign random 64-bit r_x , $H(M) = \sum_x \text{cnt}_M(x) r_x$; insert/delete x : $H \ += r_x$ / $H \ -= r_x$, merge: add hashes. XOR loses even multiplicities; use 2 random sums (and $|M|$) if collision matters.

- **Grid Path Blocking $\longleftrightarrow$ Dual Cut:** Blocking 4-dir path $(1, 1) \rightarrow (n, n)$ on grid is dual to finding an 8-dir connected barrier from left-/bottom to top/right (or Min-Cut).

- **Color Coding ($k \leq 5$):** Randomly color N vertices with k colors. Prob optimal k items get k distinct colors is $k!/k^k$. Repeat $O(k^k/k!) \approx 300$ times for error $< \epsilon$.

- **Dirichlet / Pigeonhole (4-Sum $a_x + a_y = a_z + a_w$):** When $N > \sqrt{2} \max A$, 2 pairs with equal sums always exist since $\binom{N}{2} > 2 \max A$. Find in $O(N^2)$ by hashing pairs.

- **Boyer-Moore Majority Vote:** Find majority element ($> n/2$ occurrences) in $O(n)$ time and $O(1)$ space with a single counter. Sperner's antichain max size: $\binom{n}{\lfloor n/2 \rfloor}$.

8.11 Checked

- **Read constraints and edge cases ($N = 0, 1$, negative, disconnected)?**

- Multiple test cases reset / clear global state properly?
- Array / 1-based vs 0-based indexing bounds?
- Integer overflow (use `long long` / `__int128`)?
- Functions missing `return` statements (Undefined Behavior)?
- Floating point precision / epsilon comparisons (`eps = 1e-9`)?
- Fast I/O included (`cin.tie(NULL)`, `'n'` instead of `endl`)?